Multigrid for Matrix-Free High-Order Finite Element Computations on Graphics Processors

MARTIN KRONBICHLER, Technical University of Munich
KARL LJUNGKVIST, Uppsala University

2

This article presents matrix-free finite-element techniques for efficiently solving partial differential equations on modern many-core processors, such as graphics cards. We develop a GPU parallelization of a matrix-free geometric multigrid iterative solver targeting moderate and high polynomial degrees, with support for general curved and adaptively refined hexahedral meshes with hanging nodes. The central algorithmic component is the matrix-free operator evaluation with sum factorization. We compare the node-level performance of our implementation running on an Nvidia Pascal P100 GPU to a highly optimized multicore implementation running on comparable Intel Broadwell CPUs and an Intel Xeon Phi. Our experiments show that the GPU implementation is approximately 1.5 to 2 times faster across four different scenarios of the Poisson equation and a variety of element degrees in 2D and 3D. The lowest time to solution per degree of freedom is recorded for moderate polynomial degrees between 3 and 5. A detailed performance analysis highlights the capabilities of the GPU architecture and the chosen execution model with threading within the element, particularly with respect to the evaluation of the matrix-vector product. Atomic intrinsics are shown to provide a fast way for avoiding the possible race conditions in summing the elemental residuals into the global vector associated to shared vertices, edges, and surfaces. In addition, the solver infrastructure allows for using mixed-precision arithmetic that performs the multigrid V-cycle in single precision with an outer correction in double precision, increasing throughput by up to 83%.

CCS Concepts: • **Mathematics of computing → Partial differential equations**; *Mathematical software performance*;

Additional Key Words and Phrases: Finite element method, sum factorization, matrix-free method, geometric multigrid, CUDA

ACM Reference format:
Martin Kronbichler and Karl Ljungkvist. 2019. Multigrid for Matrix-Free High-Order Finite Element Computations on Graphics Processors. *ACM Trans. Parallel Comput.* 6, 1, Article 2 (May 2019), 32 pages.
https://doi.org/10.1145/3322813

This work was supported by the German Research Foundation (DFG) under the High-Order Discontinuous Galerkin for the Exa-Scale (ExaDG) project within the Software for Exascale Computing (SPPEXA) priority program, grant agreement KR4661/2-1.
Authors' addresses: M. Kronbichler, Institute for Computational Mechanics, Technical University of Munich, Boltzmannstr. 15, Garching, 85748, Germany; email: kronbichler@lnm.mw.tum.de; K. Ljungkvist, Department of Information Technology, Uppsala University, Box 337, Uppsala, 75105, Sweden; email: karl.ljungkvist@it.uu.se.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2019 Association for Computing Machinery.
2329-4949/2019/05-ART2 $15.00
https://doi.org/10.1145/3322813

1 INTRODUCTION

Today, many-core processors, such as graphics processing units (GPUs) or the Intel Xeon Phi, are an integral part of high-performance computing and the hardware of choice in many new TOP500 supercomputers. Therefore, the development of algorithms for these throughput-oriented architectures is vital to ensure good utilization of the computational resources today and in the foreseeable future.

In the context of finite element methods with polynomials of higher degrees p, matrix-based schemes become inefficient due to a relatively dense coupling between the degrees of freedom (DoF), resulting in an evaluation complexity of $O(p^d)$ per matrix row for d-dimensional problems. In iterative solvers, the matrix-vector product can be replaced by evaluating the discretized differential operator in terms of the underlying integrals on the fly, i.e., in a matrix-free fashion. Combined with sum-factorization techniques on quadrilateral and hexahedral element shapes to interpolate between solution coefficients and values and derivatives at quadrature points, respectively, the evaluation cost can be lowered to $O(dp)$ arithmetic operations per unknown. These techniques have their origin in the spectral element community [60, 61, 72] and have been applied to complex curved geometries and variable coefficients, most often with polynomial degrees between 5 and 10 [15, 36, 42]. Later, matrix-free evaluation with sum factorization has been shown to much better align with the characteristics of modern multicore and many-core processors, making them preferable over sparse matrices already for as low a number of polynomial degrees as 2 [9, 45, 48]. This is because the matrix-free approach leads to a lower memory transfer than a sparse matrix for $p \geq 2$ [45], which translates to faster execution because matrix-vector products are bandwidth bound on all modern processors [26]. Furthermore, sum factorization is not tied to the collocation setting common in spectral elements where the nodes of Lagrange polynomials coincide with the quadrature points, providing similar efficiency also for general bases or when using more quadrature points than basis functions, because the additional computations can partly be hidden behind the unavoidable memory transfer from vectors and the index access [18, 46]. The matrix-free evaluation techniques used in this work have lately gained attention through the CEED co-design effort [10, 12, 21]. In particular, high-performance implementations with tensorial evaluation by specific tuning techniques were developed for GPUs [49, 50, 63, 68].

Previous matrix-free finite-element algorithms on GPUs have primarily focused on explicit time stepping of hyperbolic problems, e.g., in seismic wave propagation [39, 41], or on simple iterative solvers. Stronger solvers compatible with GPUs, such as multigrid methods, are currently mostly restricted to the finite-volume context [56], to finite-difference methods [19], or to matrix-based finite-element methods using low polynomial degrees [22]. Generic assembly tools complementing matrix-based solvers on GPUs have been presented in Knepley and Terrel [40]. In Fabien et al. [16], a multigrid method for the hybridizable discontinuous Galerkin method was ported to the Intel Xeon Phi, using matrix-free evaluation of the matrix-vector product in terms of some element-local matrices before hybridization. The present work goes in a similar direction but specializes on a different matrix-free setup with sum-factorization techniques. Furthermore, we develop algorithms for GPUs within CUDA covering the particular aspects of adaptively refined meshes with hanging nodes, where constraints are added to the formulation to restore H^1 conformity.

Besides presenting the algorithmic components for matrix-free geometric multigrid solvers on the GPU, the present work evaluates the performance of the GPU implementation against highly competitive CPU implementations [21, 45, 47, 48] run on a dual-socket Intel Broadwell CPU setup and an Intel Xeon Phi. The focus of this work is on the node-level performance, as the massively parallel case of multigrid on thousands of nodes is well understood [23]. The algorithms analyzed

in this work use the infrastructure of the `deal.II` finite element library [5] and the numerical test cases are available as open source software.[1]

This work is structured as follows. In Section 2, we describe the multigrid algorithm. The adaptations of the relevant ingredients for execution on GPUs are described in Section 3. In Section 4, we then investigate the usage of single-precision arithmetic for the multigrid solution of the linear system obtained from discretization of the Poisson problem. Section 5 contains our numerical experiments that verify the efficiency of the proposed algorithms. An analysis of the central components in the multigrid solver with focus on the matrix-vector product is provided in Section 6, before we conclude in Section 7.

2 MULTIGRID ALGORITHM

Geometric multigrid methods are among the most efficient solvers for linear systems arising from discretizing elliptic partial differential equations [23], offering an optimal complexity $O(n)$ in the number of unknowns n [71]. Multigrid methods combine simple iterative schemes that are effective in reducing the high-frequency error content to a given mesh resolution on a hierarchy of coarser meshes, thus attacking error components of all frequencies with optimal methods. Although multigrid methods were originally developed for elliptic equations on uniform meshes, they have been extended to the case of adaptively refined meshes [8] and to more complex applications in the past decades [54, 64, 71]. The prototypical elliptic equation is Poisson's equation, which is important, e.g., in high-Reynolds number incompressible flow discretized in time with splitting methods. For example, see Deville et al. [15] for a general overview and Fehn et al. [17] for an evaluation on contemporary hardware.

For multigrid on adaptive meshes with hanging nodes that are organized in tree-like data structures, two approaches can be distinguished. In so-called global coarsening methods [7, 34, 55, 64, 66, 67], the final mesh with cells on different logical levels in the trees also corresponds to the finest mesh of the multigrid hierarchy. All cells are coarsened simultaneously, translating to a mesh with hanging nodes also on the coarser levels of the multigrid hierarchy. The local smoothing approach [8, 31, 35], used, e.g., in Chombo [3] and similar packages, performs smoothing level by level, thus skipping less refined parts of the mesh on the finest levels. The latter method has the advantage of always smoothing on meshes without hanging nodes and a somewhat lower computational complexity. Yet the load balancing is more challenging as the mesh and unknown structure for the solution more strongly deviates from the multigrid levels. Due to the availability of a generic framework of local smoothing [31, 35] in the `deal.II` finite element library [5], local smoothing is chosen throughout this work.

In local smoothing, the residual on level l is smoothed by a discretization of the PDE subject to zero Dirichlet constraints not only on the Dirichlet boundaries of the computational domain but also on the interfaces of cells with a different refinement level, called *refinement edges* in the following. If we split the DoF associated with the cells on level l into the ones at refinement edges u_l^{E} and the interior ones u_l^{S}, the associated matrix system $A_l u_l = b_l$ can be partitioned as follows:

$$\begin{pmatrix} A_l^{\mathrm{SS}} & A_l^{\mathrm{SE}} \\ A_l^{\mathrm{ES}} & A_l^{\mathrm{EE}} \end{pmatrix} \begin{pmatrix} u_l^{\mathrm{S}} \\ u_l^{\mathrm{E}} \end{pmatrix} = \begin{pmatrix} b_l^{\mathrm{S}} \\ b_l^{\mathrm{E}} \end{pmatrix}. \tag{1}$$

For smoothing on level l, only the contributions from A_l^{SS} are considered through the matrix A_l^{SS}. When switching to a coarser or a finer level, the coupling matrices A_l^{SE} and A_l^{ES} need to be considered as well. The matrix A_l^{EE} boils down to exactly the contribution on the coarser side after

[1]The code for the present experiments is available at https://github.com/kronbichler/multigrid.

applying the usual H^1 regularity constraints, so it is taken care of on the next coarser level $l-1$ and ignored on level l. We refer to Janssen and Kanschat [31, Sec. 2] for details.

The multigrid algorithm first copies the vector entries from the active mesh cells, i.e., the terminal cells in the trees that are not further refined and carry the actual unknowns, to the respective vectors on the various levels through an indirect addressing step. On meshes that do not have hanging nodes, i.e., where all active cells are on the finest level L, this operation reduces to a plain copy operation. The multigrid V-cycle is invoked by recursive calls into level l, starting from the finest level L, as described in Algorithm 1. The prolongation is implemented by the usual polynomial interpolation from coarser mesh cells to their 2^d children, and the restriction is defined as the transpose of prolongation. Once the V-cycle is complete, the vectors with the level contributions are copied back to the single vector holding the actual DoF, i.e., the transpose of the initial copy operation into the multigrid levels.

ALGORITHM 1: Multigrid V-cycle on level l: input right-hand side b_l and initial solution u_l, output u_l

If level $l = 0$, return the result $u_0 = A_0^{-1} b_0$ of the coarse grid solver, else

- Presmoothing u_l^{S} on level l subject to the residual $r_l = b_l^{\mathrm{S}} - A_l^{\mathrm{SS}} u_l^{\mathrm{S}}$;
- Compute the residual on level l through the matrix-vector product
$$r_l = \begin{pmatrix} r_l^{\mathrm{S}} \\ r_l^{\mathrm{E}} \end{pmatrix} = \begin{pmatrix} b_l^{\mathrm{S}} \\ 0 \end{pmatrix} - \begin{pmatrix} A_l^{\mathrm{SS}} \\ A_l^{\mathrm{ES}} \end{pmatrix} \left(u_l^{\mathrm{S}} \right), \tag{1}$$
which includes the contribution on edges of different refinement level;
- Restrict the residual r_l from level l to level $l-1$ and add into initial right-hand side contribution b_{l-1} on active cells of level $l-1$ and zero otherwise;
- Invoke the multigrid algorithm with right-hand side b_{l-1} and zero initial solution u_{l-1} on level $l-1$ for the coarse grid correction;
- Prolongate the coarse grid correction u_{l-1} from level $l-1$ and add into solution u_l on level l; and
- Postsmoothing on level l subject to the residual $r_l = b_l^{\mathrm{S}} - A_l^{\mathrm{SS}} u_l^{\mathrm{S}} - A_l^{\mathrm{SE}} u_l^{\mathrm{E}}$.

In this work, we consider two flavors of the multigrid method. The first flavor is to apply a so-called full multigrid (FMG) cycle, which starts the solution process from the coarsest level and then traverses to finer levels. The algorithm is described in Algorithm 2, assuming that a consistent right-hand side vector has been computed for all levels or obtained by suitable restriction of the fine-level right-hand side vector. The FMG cycle utilizes the fact that the solution computed on a coarser mesh is already a good approximation to the solution on the fine mesh. In a recursive application of this strategy, the coarser levels are visited more often, but only a small number of multigrid V-cycles, such as $s = 1$ or $s = 2$, needs to be performed for the finest mesh where the matrix-vector products and smoother operations are most expensive. FMG is an essential ingredient to what is considered textbook multigrid efficiency, i.e., a method whose cost is only a small multiple of the cost of a matrix-vector product on the finest level [69], typically 10 (also see the recent analysis in Adams et al. [4] and Brown et al. [11]). In the second flavor, we consider the multigrid V-cycle as a preconditioner to a conjugate gradient (CG) iterative solver, which often increases the solver robustness, particularly for problems with variable coefficients, for anisotropies in the mesh, or for higher polynomial degrees [67, 71], at the cost of a considerably larger number of matrix-vector products on the fine level.

To leverage matrix-free operator evaluation techniques based on cell-wise integrals, the choice of smoothers is more restricted as compared to matrix-based computations, and effective schemes such as the Gauss–Seidel relaxation method or incomplete factorizations are not available because these directly rely on substitutions involving matrix entries or the underlying stencils [24]. In this

ALGORITHM 2: FMG cycle as a solver

- Invoke coarse grid solver, $u_0 = A_0^{-1} b_0$;
- For $l = 1, \ldots, L$ do
  - —Apply inhomogeneous Dirichlet boundary conditions on level $l - 1$;
  - —Prolongate solution including inhomogeneous boundary conditions to level l, $u_l = P_{l-1}^{l} u_{l-1}$;
  - —Compute residual $r_l = b_l - A^l u_l$;
  - —Compute correction c_l by s iterations of V-cycle(l, r_l, 0) with right-hand side r_l; and
  - —Correct solution $u_l = u_l + c_l$;

work, we restrict ourselves to a simple yet effective scheme that considers a polynomial variant of the Jacobi smoother, namely a Chebyshev iteration that only requires the action of the matrix-vector product in addition to the inverse of the matrix diagonal [1]. This is particularly interesting in the matrix-free context (see the analysis and performance model in Kronbichler and Wall [48] and May et al. [54] and references therein) and consists of two main steps, namely

- the matrix-vector product and
- vector updates, including the multiplication by the inverse of the matrix diagonal (Jacobi method).

The Chebyshev iteration needs an estimate of the largest eigenvalue $\bar{\lambda}_{\max,l}$ of the Jacobi-preconditioned matrix on each level l, which is computed before solving via a CG method with 15 iterations for all experiments reported in this study. The initial guess for the Lanczos iteration is the vector $b = (-5.5, -4.5, \ldots, 4.5, 5.5, -5.5, -4.5, \ldots)^{\mathsf{T}}$, a vector of zero mean and filled with reasonably high-frequency content that is robust against different numbers of processors and the particular mesh setup. The parameters of the Chebyshev smoother [1] are then selected to target all error components corresponding to eigenvalues $[0.06\bar{\lambda}_{\max,l}, 1.2\bar{\lambda}_{\max,l}]$, where the number 1.2 is a safety factor that allows for some inaccuracies in the eigenvalue estimate. The modes associated to eigenvalues $\lambda \leq 0.06\bar{\lambda}_{\max,l}$ will not be significantly altered but are taken care of by the coarser mesh levels recursively. The Chebyshev smoother slightly deteriorates for higher polynomial degrees p; Sundar et al. [67] reported that 14 conjugate iterations with Chebyshev (3,3) for pre- and postsmoothing are necessary to reduce the residual by a factor of 10^8 for a 3D constant-coefficient problem with $p = 16$, as compared to 5 iterations with $p = 2$. Better robustness could be achieved by Schwarz-based smoothers derived from the fast diagonalization method for separable problems [53], i.e., the Laplacian on Cartesian meshes [51, 65]. Fast diagonalization has also been extended to problems with variable coefficients or metric terms by using the closest Cartesian approximation. Considering that fast diagonalization involves similar implementations and data access patterns as the matrix-free operator evaluation, the performance of those methods can be expected to be similar to what is reported here, albeit with somewhat better iteration counts.

As a coarse grid solver, a Chebyshev iteration is selected with the parameters and number of iterations set such that the *a priori* error estimate of the Chebyshev iteration ensures a residual reduction of 10^3, where the extreme eigenvalues are determined by an initial solution with the CG method to a tolerance of 10^{-3} in terms of the relative true residual norm. Since we consider examples where the coarse mesh consists of at most a few cells, and since polynomial degrees are moderate $p \leq 7$, up to a few dozen coarse grid matrix-vector products are needed. The results presented here generalize straightforwardly to the p-multigrid case that reduce the degree of interpolation down to linear elements [20, 29, 52]. Should the coarse problem at this stage still be large, it is possible to continue coarsening with algebraic multigrid techniques.

3 MATRIX-FREE IMPLEMENTATION ON THE GPU

In this section, we describe the building blocks for an efficient matrix-free execution of the multigrid algorithm from the previous section on GPUs by considering appropriate parallelization of each of the constituent building blocks, namely

- the Laplace operator evaluation;
- the level transfer operations of prolongation and restriction;
- element-wise and reduction-type vector operations, including multiplication with the diagonal smoother; and
- handling of boundary and hanging-node constrained DoF.

3.1 Laplace Operator Evaluation

The matrix-free evaluation of a finite element operator $v = Au = (\sum_{K \in \text{cells}} P_K^\mathsf{T} A_K P_K)u$ is done by a loop over all cells in the mesh as follows (cf. [45]):

- $v = 0$,
- Loop over cells
  (i) Gather local vector values on cell: $u_K = P_K u$,
  (ii) Apply operation locally on cell: $v_K = A_K u_K$ (without forming A_K),
  (iii) Sum results from stage (ii) into the global solution vector (scatter): $v = v + P_K^\mathsf{T} v_K$.

In this loop, the matrix P_K is defined by an index vector that represents the mapping between global vector entries and the ones on the cell, and its action is implemented by a simple indirect addressing $u_{K,i} = u(\text{index}_K(i))$. On a d-dimensional hexahedral element of polynomial degree p, the number of local entries in the vector u_K is $(p+1)^d$. The local matrix-vector product $A_K u_K$ is performed by numerical quadrature without explicitly building the matrix A_K. For the example of the Laplacian with diffusivity κ (also see (5) in the following), the i-th entry of v_K on cell K is computed by Gaussian quadrature on $(p+1)^d$ points through the following formula:

$$v_{K,i} = \int_{\Omega_e} \nabla\varphi_i(x) \cdot \kappa \nabla u_K(x) dx \approx \sum_{q=1}^{(p+1)^d} \nabla\varphi_i(\xi_q) \cdot \left(\kappa J_q^{-1} J_q^{-\mathsf{T}} \sum_{j=1}^{(p+1)^d} \nabla_\xi \varphi_j(\xi_q) u_{K,j} \right) \det(J_q) w_q. \tag{3}$$

Here, φ_j is the j-th basis function constructed as the tensor product of 1D polynomials of degree p, defined as Lagrange polynomials with the nodes in the points of the $(p+1)$-point Gauss–Lobatto quadrature formula. It is evaluated in the points of a generic quadrature formula ξ_q, in our case the tensor-product of the 1D Gaussian quadrature on $p+1$ points. The quadrature weight is denoted by w_q. The integration relies on a coordinate transformation from the physical coordinates x into the unit coordinates ξ, where J_q denotes the Jacobian of the transformation from unit to real cell evaluated in the point ξ_q, with columns representing directions of partial derivatives and rows the coordinate index. Note that for problems that involve second-order derivatives on shape functions, e.g., schemes with residual-based stabilization, additional derivative terms need to be computed, increasing both the arithmetic cost and memory transfer of the matrix-free operator evaluation somewhat.

The two summations in Equation (3) correspond to matrix-vector products of size $d(p+1)^d \times (p+1)^d$ for the sum over j and and $(p+1)^d \times d(p+1)^d$ for the sum over q when testing with all test functions φ_i. In sum factorization, the Kronecker structure of these matrices due to tensor product shape functions is utilized by factoring out common coefficients. This reduces the work to a series of d 1D interpolations over all points, so-called tensor contractions. In our case, the 1D interpolations are implemented through a transformation to the collocation basis, i.e., the

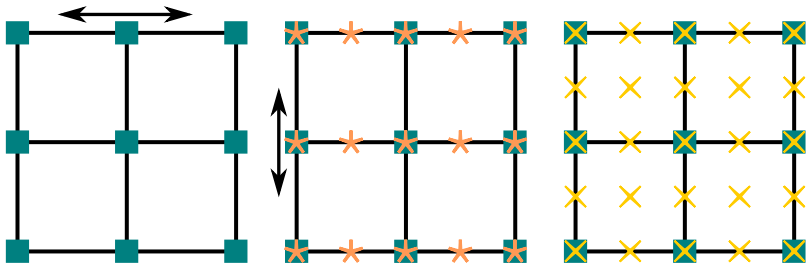


Fig. 1. We perform the prolongation by first interpolating the coarse cell node values (dark turquoise squares) along the first dimension, producing some intermediate values (orange five-armed stars), and then interpolate along the second dimension to get the fine-level node values (yellow crosses).

Lagrange polynomials defined in the points of the quadrature formula used for integration, and a derivative in the collocation space. With this layout, the evaluation of the cell Laplacian involves $4d$ tensor contractions with $(p+1)^{d+1}$ multiplications and additions each (e.g., see Kopriva [42] and Kronbilcher and Kormann [46] for details).

Our GPU parallelization [50] exploits two levels of parallelism in the matrix-free operator evaluation. First, the loop over the cells in the computational mesh runs in parallel. Second, we also parallelize within each cell with one thread per local DoF or quadrature point. This parallelization gives a sufficient degree of parallelism for higher-order elements and keeps the temporary data from the tensor evaluation in fast (register) memory. On the GPU architecture, this is realized by assigning one or more cells to each CUDA block, with the number of cells per block depending on the element order and space dimension, as to achieve large enough CUDA blocks. To optimize for coalesced memory access, we store all data at quadrature points in a structure-of-array fashion such that, e.g., Jacobians are stored each component at a time. For affine meshes where the Jacobian is constant within each cell, we only store a single value per cell, saving considerable storage space and bandwidth. We note that all Cartesian mesh cases considered in this work use this optimization. However, further arithmetic optimizations possible on Cartesian cells, such as skipping cross terms or using the tensor product of the final operator rather than integration, are not used. For the evaluation of the Laplacian, we store an effective symmetric tensor $\kappa \det(J_q) w_q J_q^{-1} J_q^{-\mathsf{T}}$, i.e., six values per quadrature point.

The CPU implementation parallelizes the loop over the mesh with domain decomposition through the message passing interface (MPI). Furthermore, vectorization batches the work of several cells together, which has been shown to considerably outperform vectorization within cells in a recent performance analysis [46, Sec. 3.3].

3.2 Multigrid Transfer Operations

For the intergrid transfer operations, a matrix-free implementation with similar ingredients as for the Laplace operator evaluation is used. The result is computed cell by cell by looping over the parent-level cells and a series of tensor contractions of the input DoF, with the important difference that the size of the output differs from the size of the input. For prolongation of the $(p+1)^d$ node values of a parent cell to those of its four (2D) or eight (3D) children, the usual polynomial interpolation on a finer set of points is used. The situation for the case of Q_2 elements in 2D is illustrated in Figure 1. Restriction is defined by the transpose of prolongation.

We employ a parallelization with one thread per coarse-level DoF, yielding the same thread block size as for the Laplace operator. We found that this approach gives a good balance between parallelism and task size. This parallelization also simplifies synchronization because there is at

Table 1. Estimate of the Share of Idle Threads Averaged Over All Contractions in Multigrid Transfer

	Two Space Dimensions				Three Space Dimensions			
Degree	1	2	3	4	1	2	3	4
Prolongation	34.4%	23.6%	18.0%	14.5%	42.2%	29.8%	23.0%	18.7%
Restriction	12.5%	8.33%	6.25%	5.00%	22.9%	15.7%	12.0%	9.67%

most one thread per output. Considering that the consecutive tensor contractions change the size of the DoF tensor one dimension at a time, each thread is responsible for computing between one and 2^d DoF throughout the computation. In addition, due to the odd number of DoF per dimension on the fine level, the last thread index in each dimension will own one rather than two points on the fine level. Table 1 quantifies the proportion of idle threads in this scheme, showing that the overhead is relatively low. In applications involving, e.g., the FMG cycle according to Algorithm 2, the suboptimal thread utilization is typically not visible because the grid transfer operations are performed less often than the matrix-vector product. If using a Chebyshev smoother of degree 3, six operator evaluations are performed in the smoother and one additional for computing the residual before restriction, compared to one restriction and two prolongation operations.

To enable sum-into semantics that are the appropriate format for cell-based evaluations, the fine-level DoF are weighed by the valence degree. As a result, the multigrid transfer can run through the same loop infrastructure over cells as the matrix-vector products, using the mechanisms from Section 4 for avoiding race conditions.

3.3 Vector Operations

For the global vector operations such as vector addition, multiplication by a scalar, and multiplication by a diagonal matrix, the action on each vector element is independent and can be performed in parallel. On GPUs, it is typically desirable to use fine-grain parallelism with a large number of concurrent threads per streaming multiprocessor (SM). However, since the warp scheduler can issue at most two independent instructions for each group of four warps per cycle, a moderate level of ILP is still necessary to achieve maximum utilization (see the GTC presentation by Volkov [74] or Section 1.4.1 in the Kepler Tuning Guide [59] for more details). Consequently, we let each thread handle a whole chunk of vector elements, which are read in a strided fashion to ensure coalescing. In our experiments, a chunk size of eight elements combined with a block size of 512 threads yielded maximum bandwidth utilization of at least 75%, both for double and single precision.

For inner products, we use the parallel reduction techniques discussed in Harris [28]. Specifically, after having performed the initial local reduction, the values inside a thread block are summed in parallel using shared memory. Eventually, the final values for each block are accumulated to global memory using atomic additions. Just as for the element-wise operations, we let each thread take care of a chunk of vector elements, and with the same chunk and block sizes an equivalent bandwidth utilization is attained.

3.4 Handling Constraints

The Laplace operator on the terminal (active) cells involves hanging node constraints that must to be eliminated to maintain H^1 regularity of the tentative solution. This is done by forcing the solution representation on the refined side to be along the polynomial representation of the coarse side, expressing the DoF on the refined side by the DoF on the coarse side through constraints. Furthermore, Dirichlet boundary conditions that are imposed in strong form also give rise to

constraints. For matrix-based methods, the contribution of constrained DoF is condensed out of the linear system in terms of the underlying interpolation to the coarse neighbor's matrix and vector entries. This elimination of a contribution initially computed by a cell integral on the refined side is done by local Gaussian elimination. In our implementation in `deal.II` [6], the constrained DoF are kept in the linear systems to maintain a single numbering of the unknowns. A dummy diagonal matrix is added to the diagonal of the system matrix in constrained rows to ensure that it remains invertible. Considering that we use the matrix diagonal within the Chebyshev smoother, this setting adds eigenvalues of unit size, making sure that the eigenvalue spectrum of the Jacobi-preconditioned matrix is not altered.

Using the preceding matrix-free setup, the matrix-vector product with n_c constraints eliminated from a system with n DoF in total can be expressed as

$$\begin{pmatrix} \begin{pmatrix} I_{n-n_c} & C^\mathsf{T} \end{pmatrix} \begin{pmatrix} \sum\limits_{K\in\text{cells}} P_K^\mathsf{T} A_K P_K \end{pmatrix} \begin{pmatrix} I_{n-n_c} & 0 \\ C & \\ & I_{n_c} \end{pmatrix} \\ 0 \end{pmatrix} \begin{pmatrix} u_{\text{unconstrained}} \\ u_{\text{constrained}} \end{pmatrix}, \tag{4}$$

where I_{n-n_c} is an $(n-n_c)\times(n-n_c)$ unit matrix and C is the matrix expressing the constrained DoF in terms of other unknowns or with empty rows for Dirichlet boundary values. In this matrix, we assume the constrained DoF to be numbered such that they appear last in the system, but the extension to the general case is straightforward: the evaluation of the products $I_m u_m$ is implemented by a simple filter operation using indirect addressing.

In matrix-free methods, matrix C for faces at hanging nodes is applied on the fly while reading the unknowns on the cells with the matrix P_K according to Kronbilcher and Kormann [45]. On the GPU, the interpolation step $P_K C$ is implemented in terms of tensor products as follows. First, for each face with a coarser neighbor, the vector entries from the coarse side are loaded according to a modified index set formed by the product $P_K C$ rather than the ones from the refined side. Subsequently, we apply an interpolation with appropriate interpolation matrix to obtain the actual values conforming to the continuity constraints. A similar (transposed) approach is used when storing the results of integration back into the result vector with $C^\mathsf{T} P_K^\mathsf{T}$. When applying the interpolation, the tensor product structure is utilized to interpolate in one coordinate direction at a time. This $(d-1)$-dimensional operation is similar to face integration applied in discontinuous Galerkin methods [46] and results in a complexity of $(p+1)^d$ operations for each constrained face. A fine-grain parallelization of the interpolation tensor contraction in analogy to the bulk of the cell integrals is used (also see Section 3 in Ljungkvist [50]).

For the matrix-vector product on the multigrid levels including interfaces of different refinement level according to (2), a similar strategy as (4) is used, namely

$$\begin{pmatrix} \begin{pmatrix} I_{n-n_{\text{boundary}}} & 0 \end{pmatrix} \begin{pmatrix} \sum\limits_{K\in\text{cells}} P_K^\mathsf{T} A_K P_K \end{pmatrix} \begin{pmatrix} I_{n-n_{\text{interface+boundary}}} & 0 \\ 0 & \end{pmatrix} \\ 0 \end{pmatrix}.$$

This method filters away a different set of DoF in the split equation system (2) on the domain and range spaces, respectively. Both the vector entries subject to Dirichlet boundary conditions and the interfaces of different refinement level are set to zero for the domain space, whereas only the vector entries associated with Dirichlet boundaries are touched for the range space. To implement this algorithm on the GPU, we keep two lists of Dirichlet-type constraints: one holding only those DoF that are located on Dirichlet boundaries of the computational domain, and a second one containing both the DoF on Dirichlet boundaries and the refinement edges. A parallel CUDA kernel then filters away the respective vector entries before and after the matrix-vector product.

4 EXPLOITING FLOATING-POINT PRECISION AND METHODS FOR CONFLICTING WRITES

For bandwidth-bound algorithms, such as the ones under consideration in this work, there are two main reasons for doing all or parts of the computations in single-precision arithmetic. If most of the bandwidth is spent on reading and writing floating-point numbers, it is clear that half as much memory will double throughput, assuming that the compute part is cheap enough. Another major advantage of using single precision is that the footprint of each thread is reduced by half and instruction latency is lower on GPUs [33]. This implies both a smaller register file and less shared memory, which allows the scheduler to fit more threads into an SM concurrently, yielding a better occupancy. For CPUs, more values will fit in caches and vector registers, producing a similar effect. We emphasize that the chosen CPU implementation uses vectorization over several elements that results in a similar cache pressure of the local data in stage (ii) of the algorithm in Section 3.1 for both single precision and double precision but processes twice as many cells at once in single precision through wider SIMD vectors.

4.1 Resolving Conflicting Writes

When running a parallel for loop over all cells of the algorithm in Section 3.1, the summations of the local results into the global result vector according to stage (iii) can potentially lead to a race condition. This is because vector entries associated to shared entities, such as mesh vertices or mesh edges, will receive partial integral contributions from several cells. The $n_{\text{cells}}(p+1)^d$ integrals that are computed get written into approximately $n_{\text{cells}}p^d$ unique vector entries, with a relatively low number of conflicts. In Ljungkvist [50], this issue was addressed by a method based on graph coloring (also see Kormann and Kronbichler [43], Kronbichler et al. [47], and Turcksin et al. [73], in which concurrent updates are avoided by only processing safe combinations of elements simultaneously). In this article, we develop a new method using atomic operations. On earlier generations of Nvidia GPUs, atomic operations incurred a substantial overhead, but starting with the Kepler architecture, these are performed by a single instruction with a "fire-and-forget" semantic. This means that the instruction returns immediately and conflict resolution is taken care of by the cache system (see Section 5.2.11 in Wilt [75] for more details). Moreover, the Pascal architecture extends support for fast atomics to double-precision values [58].

To compare the performance using single and double precision, respectively, we run a microbenchmark involving the matrix-free Laplace operator evaluation described in Section 3. We consider the 3D shell example described in Section 5.2, as well as a similar 2D variant on an annulus; apply the corresponding operator repeatedly to a vector; and compute the average time of 100 evaluations. As mentioned earlier, the switch to single precision can affect the choice between atomic operations or graph coloring for conflicting updates depending on the GPU architecture, so we run the benchmark for both the graph coloring and the atomics approach.

Table 2 shows that single precision is substantially faster than double precision. On the Pascal-generation Nvidia Tesla P100, performance is higher by 43% and 100%, often close to the theoretical factor of 2 due to twice the lanes. Furthermore, atomics are the fastest synchronization method on the P100 across the whole range of experiments, which was not the case for the older Kepler generation GPU K40, where coloring was faster in double precision except for $p = 1$ in 2D due to the lack of single-instruction double-precision atomics. Moreover, although this study focuses on the solve part, we note that atomics have the additional benefit of halving the setup time by avoiding the expensive graph coloring.

Table 2. Runtime for a Single Laplace Operator Evaluation on a Shell Geometry for Atomics and Coloring

				Time P100 (ms)				Time K40 (ms)			
				Double		Single		Double		Single	
		Cells	DoF	Color	Atom	Color	Atom	Color	Atom	Color	Atom
2D	$p = 1$	10M	10M	8.41	5.05	5.17	2.74	26.24	24.18	16.02	10.26
	$p = 3$	2.6M	24M	9.45	5.70	6.59	3.89	30.67	41.14	20.29	16.97
3D	$p = 1$	1.6M	1.6M	3.18	1.61	1.77	0.904	12.88	18.68	7.83	6.38
	$p = 3$	0.20M	5.4M	3.24	2.39	1.97	1.24	12.35	15.45	8.16	7.21

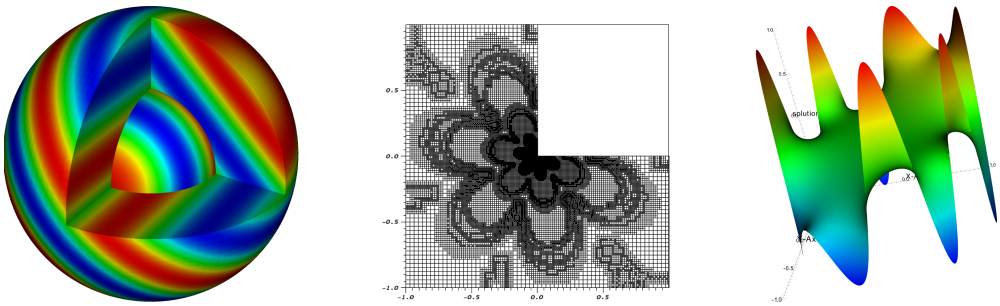
Table 3. Comparison of Mixed and Full Double Precision for Solving the Variable-Coefficient Laplacian on an Annulus (2D) and Shell (3D) Geometry With the CG Method Preconditioned by a Multigrid V-cycle Using Atomics and Running on the Nvidia Tesla P100

				Double Precision			Mixed Precision		
		Cells	DoF	L_2 Error	CG Iterations	Time (s)	L_2 Error	CG Iterations	Time (s)
2D	$p = 1$	10M	10M	3.31e-06	8	1.47	3.31e-06	8	0.964
	$p = 3$	2.6M	24M	1.49e-07	8	2.00	1.49e-07	8	1.29
3D	$p = 1$	1.6M	1.6M	2.89e-03	14	0.817	2.89e-03	14	0.556
	$p = 3$	0.20M	5.4M	3.26e-06	16	1.43	3.26e-06	16	0.781

4.2 Exploiting Single Precision in Multigrid

For certain wave propagation problems which can be integrated explicitly in time, the effect of precision on the solution accuracy is relatively weak, and single precision arithmetic has been demonstrated to work successfully [41]. When solving linear systems of equations, either iteratively or by direct methods, using high enough floating point precision is important for accurate results due to the accumulation of rounding errors. Göddeke et al. [25] find that for the linear system arising from a finite-element discretization of an elliptic PDE, single precision used throughout is typically not enough to produce an acceptable accuracy in the solution. For example, the convergence control in an iterative CG solver breaks down. Even though progress appears to be made when looking at the solver's representation of the residual, the true residual is not shrinking, and the L_2 error actually grows. Instead, Gropp et al. [27] and later Göddeke et al. [25] propose a mixed-precision approach and show that an outer double-precision CG iteration combined with a single-precision multigrid V-cycle as a preconditioner preserves the accuracy and iteration numbers of a full double-precision reference. In our implementation, the V-cycle is fully done in single precision, and the format is converted on entry into the V-cycle both in the case of the CG method (where single precision represents a preconditioner) and the FMG cycle of Algorithm 2 (where each fine level only adds a correction to the solution from the coarser level).

In Table 3, we compare a CG method run in double precision preconditioned by a multigrid V-cycle with a Chebyshev iteration of degree (6,6) for pre- and postsmoothing in single precision against doing all computations in the V-cycle in double precision. The problem setup is according to Section 5.2. The CG algorithm is terminated once the l_2 norm of the unpreconditioned residual $b - Ax$ has decreased by 10^9 compared to the l_2 norm of the right-hand side b. We find that the mixed precision approach speeds up the solution of the linear system by 47% to 83%, whereas the final L_2 discretization error is the same irrespective of whether the V-cycle is done in single precision or in double precision. This is because the outer matrix-vector product in the CG method is done in double precision and controls the convergence. The single-precision computations of the V-cycle add corrections that are significant to up to two digits (also see the multigrid convergence



(a) (b) (c)

Fig. 2. (a) Solution of the 3D shell problem for a mesh with 0.2 million elements of degree $p = 2$ and 1.6 million DoF (Section 5.2). (b) Adaptively refined mesh for the 2D L-shaped domain after four refinement cycles (Section 5.3). (c) Solution of the 2D minimal surface problem for a mesh with 82,000 elements of degree $p = 4$ and 1.3 million DoF (Section 5.4).

rates in Tables 6 and 8 in the following for the error reduction by one V-cycle). Compared to typical roundoff errors inside the V-cycle, which occur at the fifth to sixth digit for single precision, the leading order accuracy of the correction is not affected. Furthermore, roundoff errors are naturally of high frequency that are strongly damped by the multigrid smoother and thus prohibited from growing to the significant digits during the course of the multigrid iteration. Finally, the algebraic iteration error is smaller than the discretization error for three of the four cases, further reducing the impact of the precision within the preconditioner. Only for the 2D annulus with cubic polynomials is CG tolerance not strict enough to ensure a fully converged iterative solution, resulting in an L_2 error norm of the solution that is higher than it would be in terms of discretization accuracy. Nonetheless, in isolated cases where the residual happens to come close to the termination tolerance, for example, single precision can have an influence and cause one additional iteration. These experiments extend the results of Göddeke et al. [25] to the case with a strongly varying coefficient, high-polynomial degrees, and curved geometry.

5 MULTIGRID EXPERIMENTS

To evaluate the performance of our multigrid implementation, we consider four numerical problems based on the Poisson equation

$$-\nabla \cdot (\kappa \nabla u) = f \quad \text{in } \Omega, \qquad u = g \quad \text{on } \partial\Omega, \tag{5}$$

namely

- a cube geometry with constant coefficient in Section 5.1;
- a spherical shell domain with curved geometry and widely varying coefficient in Section 5.2;
- an L-shaped domain with adaptive mesh refinement in Section 5.3; and
- a nonlinear minimal-surface equation solved with a Newton iteration in Section 5.4.

The latter three problems are sketched in Figure 2. For each problem, the GPU multigrid solver is compared to a reference implementation for CPUs. The GPU experiments use a system with an Nvidia Tesla P100 GPU, two 20-core Intel Xeon E5-2698v4 CPUs, and 512GB DDR4 memory. The CPU experiments run on 2×14 cores of an Intel Xeon E5-2690v4 processors at 2.6GHz (labeled Broadwell), and on a 64-core Intel Xeon Phi Knights Landing 7210 running at 1.1GHz (actual frequency attained in AVX-512-heavy code) (Table 4). The Xeon Phi is run with 128 threads (two-way simultaneous multithreading) in flat mode, and all matrix-free operations are served

Table 4. Properties of Hardware Under Consideration

	Broadwell E5-2690 v4	Xeon Phi 7210	P100
Number of Cores	2 ×14	64	60 SM
Arithmetic Throughput, Peak	1,164GFLOP/s	2,253GFLOP/s	4,700GFLOP/s
Memory Bandwidth, STREAM Triad	115GB/s	420GB/s (MCDRAM)	600GB/s
Nominal Power Consumption	2 ×135 W	215W	250W
Measured Power Consumption	2 ×(115–125) W	190–210W	195–225W

The power measurements on Broadwell and Xeon Phi are CPU values only via hardware counters, measured by the Likwid tool [70], and do not include the power of DRAM and other parts of the system. The numbers for P100 are board power consumption reported by `nvidia-smi`, ignoring the power consumption of the host. The data are shown as a range for matrix-vector products at various degrees $p = 1, \ldots, 7$, with higher values for larger degrees. The STREAM triad numbers for Broadwell and the Xeon Phi report values without the read-for-ownership transfer, i.e., with streaming stores.

Table 5. Exemplary Setup Costs for the Case $p = 4$ at 57 Million DoF

	Broadwell	Xeon Phi	P100
Grid Creation, Distribute DoF	3.75s	5.41s	25.1s (host, partly multithreaded)
Setup Matrix-Free Data Structures	1.21s	3.06s	3.95s (initialization on host, multithreaded) 1.28s (copy host to device)
Compute Right-Hand Side & Boundary Conditions	0.76s	0.92s	0.46s (device)
Compute Matrix diagonal	1.08s	0.67s	0.49s (device)
Chebyshev Eigenvalue Estimate	1.27s	0.80s	0.63s (device)
Cost of V-cycle With Chebyshev (4, 4)	0.56s	0.30s	0.29s (device)

from fast MCDRAM [32] by using `numactl --membind=1` to bind the program to fast memory. The CPU implementation is parallelized with MPI only, which has been found to perform better than a hybrid MPI+Intel TBB approach also on the Xeon Phi [47]. The CPU version utilizes vectorization through the AVX2 instruction set extension on Broadwell and through AVX-512 on the Xeon Phi. The CPU code is compiled with the GNU C++ compiler `g++`, version 6.2, with compiler flags `-O3 -march=native -funroll-loops`. Due to the explicit vectorization over several elements for the CPU code [45], more than 98% of arithmetic instructions are done in packed (full-lane) form. By comparing to optimized reference CPU versions, we try to present the new GPU algorithm as fairly as possible. Furthermore, Table 4 reports a similar power consumption for the three architectures, which allows associating faster performance also with better energy efficiency.

Table 5 lists the setup costs exemplarily for the case $p = 4$ on a Cartesian mesh according to Section 5.1. The cost for grid generation and distribution of DoF is governed by the data structures in the `deal.II` library, which are fully parallelized with MPI for Broadwell and the Xeon Phi but only partly multithreaded for the GPU experiment (it would be possible to use MPI parallelism on the CPU as well, however). The evaluation of the metric terms and extraction of DoF indices for the matrix-free evaluation framework are considerably cheaper and well threaded. Regarding the transfer from the host to the device, about 25% of costs are due to copying the solution and right-hand side vectors, and the rest is due to the indices for DoF in single and double precision, constraint lists, and the (compressed) geometry. In many finite element computations, particularly for nonlinear or time-dependent problems where many systems need to be solved, the setup is done only once (or only when adapting the mesh) and thus much less frequently than the other operations. The remaining operations can be done completely on the device and have a cost similar

Table 6. Multigrid Experiments for the Constant-Coefficient Poisson Problem on a Cube, Both for an FMG Cycle Using a Degree-Specific Smoother Setup and a CG Solver Preconditioned by a Single V-Cycle of Multigrid With Chebyshev (4, 4)

Polynomial Degree p	1	2	3	4	5	6	7
No. of cells	256^3	192^3	128^3	96^3	64^3	64^3	64^3
No. of DoF	17M	57M	57M	57M	33M	57M	91M
L_2 error (FMG cycle)	1.94e-04	3.42e-06	6.92e-08	2.29e-09	1.53e-10	1.59e-12	3.49e-14
Chebyshev smoother degree (pre, post)	(2, 2)	(3, 4)	(4, 5)	(5, 6)	(3, 4)	(4, 5)	(7, 8)
No. of V-cycles in FMG	1	1	1	1	2	2	2
Reduction rate per V-cycle in FMG	0.027	0.090	0.021	0.022	0.063	0.085	0.0090
Runtime FMG Broadwell [s]	0.196	0.549	0.592	0.643	0.483	0.908	2.36
Runtime FMG Xeon Phi [s]	0.166	0.364	0.379	0.421	0.352	0.635	1.68
Runtime FMG P100 [s]	0.145	0.315	0.305	0.351	0.282	0.609	1.55
No. of PCG iterations, rel. true residual 10^{-9}	6	6	6	6	6	6	7
Reduction rate per CG iteration	0.027	0.027	0.030	0.028	0.030	0.028	0.035
Runtime PCG Broadwell [s]	1.56	3.14	2.90	2.75	1.57	2.67	4.95
Runtime PCG Xeon Phi [s]	1.52	2.15	1.89	1.75	1.16	1.90	3.51
Runtime PCG P100 [s]	1.09	1.84	1.52	1.37	0.84	1.86	3.43

to one to three multigrid V-cycles with a Chebyshev (4,4) smoother. Note that the right-hand side computation presented in Table 5 is dominated by trigonometric functions that are not specifically vectorized on Broadwell and the Xeon Phi, explaining the longer runtime of the latter.

5.1 Poisson's Equation on a Cube

As a first numerical test, we consider the Poisson equation with constant coefficient $\kappa \equiv 1$ on the cube $[-0.9, 1]^3$. In this example, the geometry is Cartesian, allowing for a compression of the stored effective geometry coefficient. Thus, the only significant memory access is due to the solution and residual vectors during the multigrid algorithm and the indices for the indirect addressing through the matrices P_K according to Section 3. The right-hand side f and the Dirichlet boundary data g in (5) are set such that the analytic solution obeys

$$u(x, y, z) = \sin(3\pi x)\sin(3\pi y)\sin(3\pi z).$$

Experiments with the FMG cycle according to Algorithm 2 and the CG method preconditioned by a V-cycle of Algorithm 1 are presented in Table 6. The Dirichlet-constrained DoF carry a zero right-hand side and residual while solving and are applied in a postprocessing step. The degrees of the Chebyshev polynomial for pre- and postsmoothing have been determined experimentally such that the L_2 error obtained with the FMG cycle is within 5% of the actual discretization error. Clearly, the work increases for higher polynomial degrees p because the residual must be reduced more strongly for methods with a higher rate of convergence. For polynomials of degree $p = 6$, for instance, the error must be reduced by a factor of 128 to retain optimal convergence rates $\mathcal{O}(h^{p+1})$. For degrees $p = 5, 6, 7$, two multigrid V-cycles, $s = 2$ in Algorithm 2, have been found to deliver better performance than more Chebyshev iterations on a single V-cycle. A single V-cycle is used within the CG solver with a fixed smoother setup of Chebyshev (4, 4) for all degrees. The chosen Chebyshev degree has been found to produce the smallest run time when averaged over $p = 1$ to $p = 7$.

The solution times listed in Table 6 and the solver throughput plotted in Figure 3, measured as the number of unknowns solved per second, show that the FMG cycle is clearly superior to the

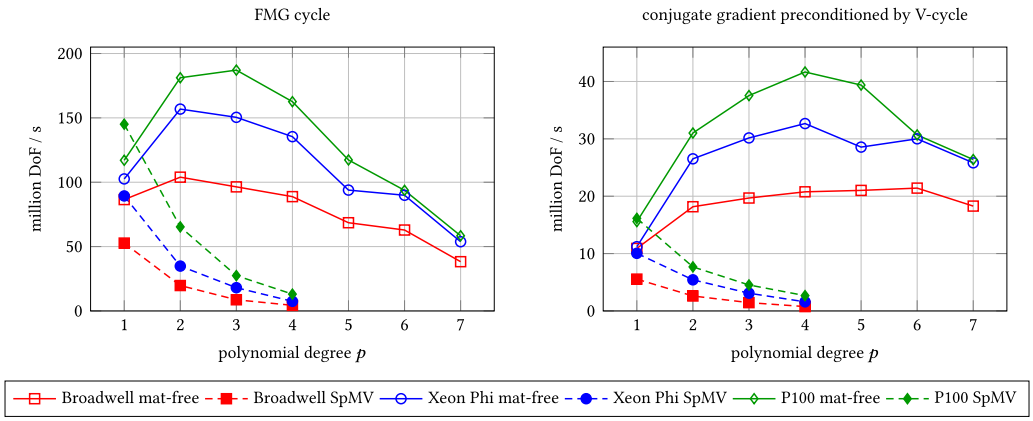


Fig. 3. Throughput measured as the number of unknowns (DoF) solved per second for the proposed matrix-free implementation and sparse matrix-vector products (SpMV) for the constant-coefficient Poisson equation on a cube with the FMG cycle (left panel) and a CG method preconditioned by a V-cycle, reducing the true residual by 10^9 (right panel).

CG method. For polynomial degrees $p \leq 4$, FMG is 4 to 9 times as fast. For $p = 4$ and 57 million DoF, the latter achieves a throughput of 89 million DoF/s on 28 Broadwell cores, 135 million DoF/s on 64 Xeon Phi cores, and 162 million DoF per second on the P100. However, the cost for solving increases for higher polynomial degrees, which is due to a higher number of Chebyshev iterations and V-cycles, respectively. Conversely, the throughput is almost independent of the polynomial degree for the preconditioned CG method, especially on the CPUs. The main reason is that the CG method is required to reduce the l_2 norm of the unpreconditioned residual $r = b - Ax$ against the l_2 norm of the initial residual, i.e., the right-hand side due to the zero start vector, by a degree-independent factor 10^9. It should be noted that for this setup and the chosen analytic solution, the CG method fails to reach the discretization accuracy for $p = 6$ and $p = 7$ and terminates with L_2 discretization errors of around 10^{-10}. Conversely, the FMG cycle can reach L_2 discretization errors as low as $3.5 \cdot 10^{-14}$ for $p = 7$, prevented from decreasing even further only by the unavoidable roundoff errors of double-precision numbers. The L_2 error norms are normalized by the volume of the domain $1.9^3 = 6.86$.

When comparing the three architectures, P100 shows the best performance. This can be explained by the more favorable hardware properties, namely the higher memory bandwidth and higher arithmetic capabilities listed in Table 4. For higher polynomial degrees, the Xeon Phi reaches a similar performance as the GPU, particularly when combined with the CG solver. This is partly caused by the non-optimal behavior of the chosen GPU threading scheme for $p \geq 6$ (see Świrydowicz et al. [68] for alternative schemes). The throughput on Broadwell is only about half as high, which is mainly due to the lower memory bandwidth, as analyzed in more detail in Section 6.

Figure 4 complements the behavior of the three architectures by a throughput-spectrum diagram for the case $p = 4$, plotting the achieved DoF/s throughput with the FMG cycle over the wall time of the solver for a series of meshes starting from a $(4 \times 4 \times 4)$ mesh with 4,913 unknowns. The meshes are generated by refinement of the three coarse meshes of size $1 \times 1 \times 1$, $2 \times 1 \times 1$, and $2 \times 2 \times 1$, respectively, doubling the number of unknowns between two data points. This diagrams shows that the Broadwell CPU is fastest for small problem sizes that can be solved in 0.02s or less. As the problem size increases, the P100 GPU and the Xeon Phi increase their throughput more strongly and eventually surpass the CPU, whose throughput is reduced once the capacity of the L3 cache

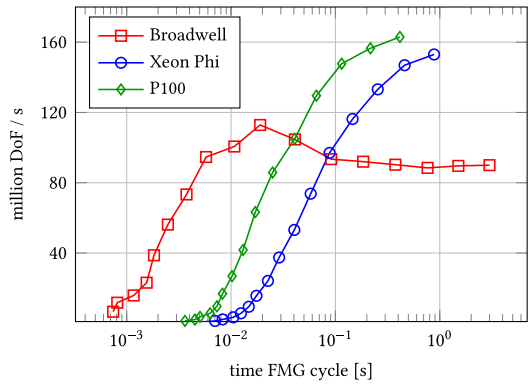


Fig. 4. Throughput of the multigrid FMG cycle over the solution time on Broadwell, the Xeon Phi, and the P100 GPU for the 3D constant-coefficient Laplacian with elements of degree $p = 4$ on a cube.

Table 7. Comparison Against HPGMG-FE on 128^3 Mesh With Elements of Degree $p = 2$, i.e., 17.0 Million DoF, on 28 Broadwell Cores

	FMG Cycle		Matrix-Vector Product		
	Time Solve	Throughput	Time	MDoF/s	Arithmetic Perf.
HPGMG-FE	0.853s	19.9MDoF/s	0.105s	161MDoF/s	186GFLOP/s
Present Code	0.250s	67.9MDoF/s	0.0323s	526MDoF/s	110GFLOP/s
Present Code, Single Precision			0.0178s	956MDoF/s	200GFLOP/s

is exceeded and data must be fetched from RAM memory. The results in Figure 4 show that the CPU is preferable for small and moderate problem sizes where latency is important, whereas the GPU and the Xeon Phi excel for large problem sizes. The crossover point between Broadwell and P100 is for a problem with 4.2 million DoF that is solved in 0.04 seconds on both architectures.

5.1.1 Comparison to HPGMG-FE. To document the quality of the CPU code used in this work, we evaluate the performance of the multigrid FMG cycle against the HPGMG-FE benchmark implementation for quadratic polynomials $p = 2$ [2]. HPGMG-FE implements an FMG cycle via a full approximation storage (FAS) scheme that is also applicable to nonlinear problems (e.g., see Trottenberg et al. [71]). The FAS cycle does an additional restriction of the solution to coarser levels within a V-cycle, one more residual on coarser levels and some additional vector updates when compared to FMG. These operations contribute with less than 5% of the total V-cycle time, enabling a direct comparison of throughput. We run the proposed solver on a variant of the Cartesian mesh problem with slightly deformed geometry to avoid the Cartesian optimizations of `deal.II` [45] and ensure better comparability to HPGMG-FE. Both codes are run with 28 MPI ranks and use instruction set optimizations for AVX2 and FMA.

The results in Table 7 document a 3.4 times higher throughput for the FMG cycle with the present solver than with HPGMG-FE. The higher performance can be explained by faster matrix-vector products and the fact that the V-cycle runs in single precision according to Section 4. The threefold throughput difference in the double-precision matrix-vector product can be traced back to differences in the arithmetic work. HPGMG performs 45 tensor contractions, 27 for evaluating the metric terms via the Jacobian J_q and 18 for the shape gradients and test functions, as compared to $12 = 4d$ for our method the basis change into the collocation basis described in Section 3.1. No

Table 8. Multigrid Experiments on the 3D Shell Benchmark With Chebyshev Smoother of Degree (4, 4) for Pre- and Postsmoothing

Polynomial Degree p	1	2	3	4	5	6	7
No. of Cells	3.1M	1.6M	197k	197k	197k	24k	24k
No. of DoF	3.2M	13M	5.4M	13M	25M	5.4M	8.5M
L_2 Error (FMG cycle)	9.99e-04	1.03e-05	3.26e-06	1.19e-07	2.87e-08	9.08e-08	1.19e-08
No. of V-Cycles in the FMG	4	6	7	9	11	13	16
Reduction Rate per V-cycle in the FMG	0.19	0.29	0.32	0.39	0.38	0.51	0.58
Runtime FMG Broadwell (s)	0.390	1.20	0.458	1.31	3.01	0.717	1.46
Runtime FMG Xeon Phi (s)	0.344	0.825	0.461	1.13	2.06	0.977	1.86
Runtime FMG P100 (s)	0.161	0.545	0.276	0.619	1.35	0.390	0.830
No. of PCG iterations, rel. true residual 10^{-9}	14	17	18	20	27	28	33
Reduction rate per CG iteration	0.21	0.28	0.30	0.35	0.45	0.47	0.47
Runtime PCG Broadwell (s)	1.30	3.40	1.15	3.04	7.39	1.56	3.04
Runtime PCG Xeon Phi (s)	0.876	1.91	1.03	1.97	4.23	1.32	2.78
Runtime PCG P100 (s)	0.522	1.34	0.687	1.38	2.65	0.856	1.73

metric terms are computed with our code, which instead loads a precomputed effective metric tensor $\kappa \det(J_q) w_q J_q^{-1} J_q^{-\mathsf{T}}$ from memory, based on the tradeoff explained in Kronbilcher and Kormann [46]. Finally, the precomputed tensor also reduces the number of arithmetic operations at quadrature points, as does the even-odd decomposition within the tensor contractions [46]. Table 7 documents a higher GFLOP/s performance for HPGMG-FE due to the additional arithmetic work, which must be put against the throughput optimizations of the present code.

5.2 Poisson's Equation on a Spherical Shell With a Variable Coefficient

In our second numerical test, we consider a spherical shell with inner and outer radii of 0.5 and 1.0, respectively. We use uniformly refined meshes with elements of polynomial degrees one to seven and solve on the largest mesh that fits in the GPU memory (Table 8). The case $p = 1$ starts from 12 cells of a rhombic dodecahedron, whereas all other cases use a coarse mesh of 6 cells forming a hexahedron. The geometry is represented in a curved way by isoparametric mappings of the same degree as the interpolation of the finite element, with auxiliary points placed according to a spherical manifold description [5]. We use a coefficient that varies over six orders of magnitude [67],

$$\kappa(\vec{x}) = \kappa(x_1, \ldots, x_d) = 1 + 10^6 \prod_{e=1}^{3} \cos^2(2\pi x_e + 0.1e)\,,$$

and let right-hand side f and Dirichlet data g be defined by constructing a problem with exact solution $u = \sin(\pi(x + y))$. Like in the previous example, the CG iteration is stopped once the l_2 norm of the unpreconditioned residual has decreased by 10^9 compared to the initial l_2 norm of the residual, i.e., the right-hand side norm. Figure 2(a) shows the solution on a section of the shell. The errors of the numerical solution reported in Table 8 are higher than for the cube, which is partly due to the coarser mesh, partly due to the influence of the strongly varying coefficient, and partly due to the representation of the curved manifold that results in errors on the same magnitude as the discretization error.

The results in Figure 5 show that P100 is 1.7 to 2.4 times faster than Broadwell for the FMG cycle and 1.7 to 2.8 times faster for the CG solver. Against the Xeon Phi, the P100 achieves a speedup of 1.5 to 2.2 for the FMG and 1.4 to 1.7 for the CG solver. Overall, both the Xeon Phi and P100 clearly outperform Broadwell due to the massively higher memory bandwidth. This factor will be more

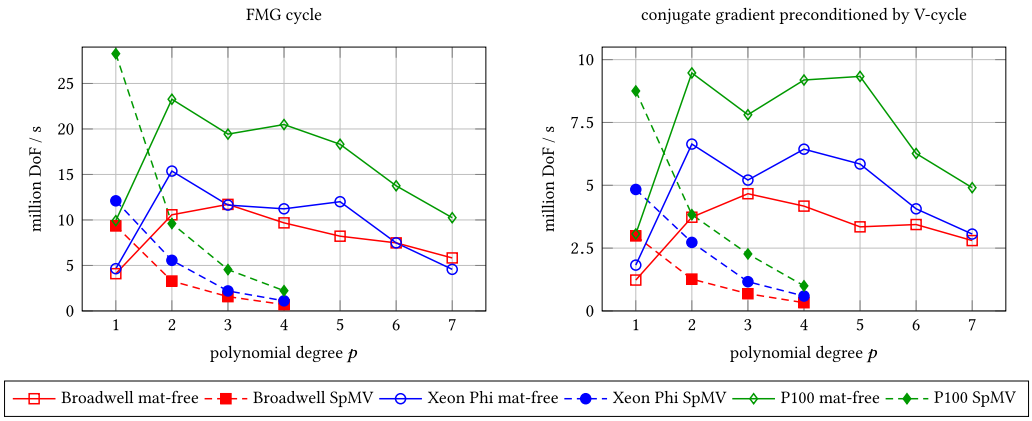


Fig. 5. Throughput for the shell domain problem with variable coefficients in three dimensions for an FMG cycle (left panel) and the CG preconditioned by multigrid (right panel).

closely analyzed in Section 6. At the same time, the performance of the Xeon Phi in the FMG case is affected more strongly by time spent on coarser mesh levels where its throughput is worse than that of Broadwell and P100. For example, the Xeon Phi is slower than Broadwell for $p = 7$ for the FMG solver, which visits coarse levels more often, whereas the Xeon Phi is about 9% faster than Broadwell for the preconditioned CG solver.

The absolute runtimes for this example are considerably higher than for the constant-coefficient problem on the cube on all three architectures. This is because of a higher number of sweeps through the data, i.e., V-cycles in the FMG case or CG iterations. This far-from-optimal behavior is expected from multigrid theory due to use of an operator-independent coarse space that poorly resolves the varying coefficient. Furthermore, the throughput of the matrix-vector product is also lower on the curved mesh because a full coefficient tensor with six values must be loaded at each quadrature point, more than doubling the memory transfer against the constant-coefficient case (see Section 6.2 for details).

Figures 3 and 5 also contain data points where the matrix-vector product is done by sparse matrix-vector products for comparison. We use the CRS format on Broadwell and the Xeon Phi (implemented in single and double precision through the Tpetra package of Trilinos [30]) and the cuSPARSE implementation on P100 [57] for the sparse matrix algebra and combine it with the same smoother setup as in the matrix-free cases. The sparse matrix-vector product involves less memory transfer than the matrix-free implementation for $p = 1$ on the mesh geometry [45] and is faster than the matrix-free solvers as a consequence. Due to the complexity $\mathcal{O}(p^3)$ per unknown, the throughput of the matrix-based scheme quickly reduces with p and is about an order of magnitude lower for $p = 3, 4$. Furthermore, high-order matrix-free implementations achieve a higher throughput per DoF than the matrix-based algorithm for linear finite elements, again implied by the faster matrix-vector product. We note that matrix-based higher-order schemes can still provide better runtimes to reach a fixed discretization accuracy as they typically need fewer DoF, as detailed in Chang et al. [13] and Kronbichler and Wall [48]. Given these results, the matrix-free results presented here further increase the attractiveness of higher-order methods.

5.3 L-shaped Domain With Singular Solution and Adaptivity

The 2D L-shaped domain defined by removing one quadrant of a square is a common test geometry for numerically solving the Poisson equation due to the singularity arising from the inner corner.

Table 9. Convergence and Computing Time for the L-Shaped Domain in 2D for Elements of Degree $p = 2$, Using a Chebyshev (6, 6) Smoother

Cells	DoF	Convergence Rate	No. of Iterations	L_2 Error	Broadwell Solve (s)	Xeon Phi Solve (s)	No. of Iterations	L_2 Error	P100 Solve (s)
3.0k	13k	0.0080	5	1.13e-04	2.85e-03	5.18e-02	5	1.13e-04	2.10e-02
5.8k	24k	0.032	7	4.41e-05	6.61e-03	7.00e-02	7	4.41e-05	3.59e-02
11k	47k	0.041	7	1.73e-05	9.72e-03	8.83e-02	7	1.73e-05	4.26e-02
21k	94k	0.040	7	6.83e-06	1.46e-02	1.05e-01	7	6.83e-06	9.95e-02
41k	0.18M	0.043	7	2.70e-06	2.34e-02	1.36e-01	7	2.70e-06	6.11e-02
78k	0.35M	0.051	8	1.07e-06	5.02e-02	1.51e-01	7	1.07e-06	7.18e-02
0.15M	0.68M	0.045	7	4.23e-07	8.53e-02	1.97e-01	7	4.23e-07	1.31e-01
0.29M	1.3M	0.043	7	1.68e-07	1.87e-01	3.38e-01	7	1.68e-07	2.35e-01
0.55M	2.5M	0.045	7	6.65e-08	4.00e-01	6.00e-01	7	6.65e-08	3.11e-01
1.1M	4.8M	0.047	7	2.65e-08	7.97e-01	1.13e+00	7	2.65e-08	5.28e-01
2.0M	9.4M	0.046	7	1.06e-08	1.68e+00	2.10e+00	7	1.06e-08	7.71e-01
3.9M	18M	0.047	7	4.26e-09	3.52e+00	4.04e+00	7	4.25e-09	1.26e+00

The singularity leads to a strong localization of errors, creating a situation that lends itself well to adaptive mesh refinement. We first consider the standard 2D L-shaped domain, and then two 3D extensions: an extrusion of the 2D case and a "hyper-L" defined by a cube with one octant removed that exhibits a typical 3D corner singularity.

To adaptively refine the mesh, we approximate the error per cell using the indicator of Kelly et al. [37]. Then we mark the ratio f_r of cells with the largest error for refinement and the ratio f_c of cells with the smallest error for coarsening. We use a refinement factor f_r of 0.30 for the 2D geometry and the extruded 3D geometry, whereas $f_r = 0.15$ is used for the hyper-L geometry. In both cases, a coarsening factor f_c of 0.03 is used. In practice, this results in roughly a doubling of the number of DoF per refinement level, when including the extra refined and coarsened cells due to the vertex level-difference limit of one in the `deal.II` library. Figure 2(b) shows an example of an adaptively refined mesh on the 2D L-shaped domain.

For the 2D case and the extruded 3D case, we consider a zero right-hand side $f = 0$ and Dirchlet conditions defined by constructing a problem with exact solution,

$$u = \sqrt[3]{x^2 + y^2} \sin\left(\tfrac{2}{3}\left(\pi + \arctan\left(\tfrac{y}{-x}\right)\right)\right).$$

For the 3D hyper-L case, we instead use homogeneous Dirichlet conditions at $x = -1$, homogeneous Neumann conditions elsewhere, and a trivial coefficient $\kappa \equiv 1$. The CG solver is terminated once the unpreconditioned residual has decreased by a factor of 10^9. As in Section 5.1, the geometry tensor is constant throughout cells, which is utilized in our implementation.

Table 9 shows that the CPU and GPU versions converge equivalently, both in terms of a constant number of multigrid iterations (up to negligible fluctuations due to small roundoff differences, which eventually lead to slightly different meshes upon the evaluation of the error estimator), and with an error that is reduced by a factor of 2.6 with each doubling of the number of DoF. Furthermore, we see that the GPU version has a slower baseline performance for small problems compared to Broadwell. The tipping point in favor of the GPU is at around 2 million DoF or 0.2 seconds as seen in the throughput-spectrum diagram of Figure 6, reaching speedups of up to 2.8 times at larger sizes. In fact, although the throughput of Broadwell reaches a plateau and even

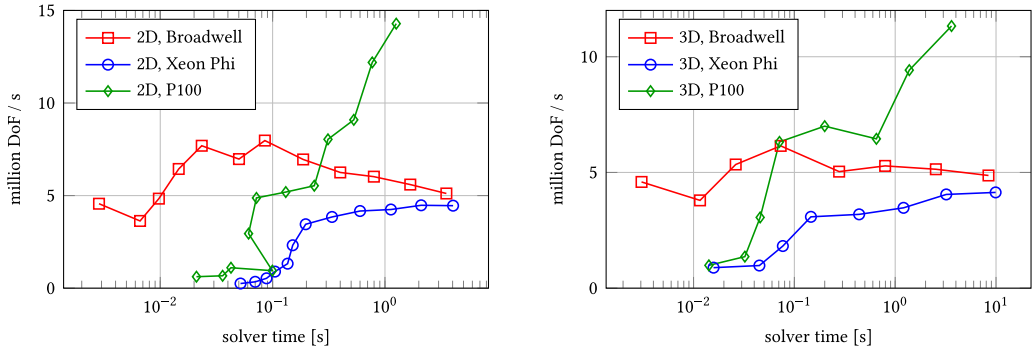


Fig. 6. Solver throughput (million DoF/s) as a function of wall time for solving the Poisson equation with $p = 2$ on the 2D and the extruded 3D L-shaped domain.

Table 10. Convergence and Computing Time for the Extruded L-Shaped Domain to 3D for Elements of Degree $p = 2$

Cells	DoF	Convergence Rate	No. of Iterations	L_2 Error	Broadwell Solve (s)	Xeon Phi Solve (s)	No. of Iterations	L_2 Error	P100 Solve (s)
1.5k	14k	0.0079	5	6.02e-04	3.05e-03	1.58e-02	5	6.02e-04	1.42e-02
4.8k	44k	0.062	8	2.22e-04	1.16e-02	4.50e-02	8	2.22e-04	3.23e-02
15k	0.14M	0.057	8	8.41e-05	2.62e-02	7.69e-02	8	8.41e-05	4.58e-02
46k	0.45M	0.069	8	3.24e-05	7.32e-02	1.46e-01	8	3.24e-05	7.12e-02
0.14M	1.4M	0.079	9	1.26e-05	2.78e-01	4.39e-01	8	1.26e-05	2.00e-01
0.44M	4.2M	0.086	9	4.93e-06	7.95e-01	1.21e+00	9	4.93e-06	6.51e-01
1.4M	13M	0.083	9	1.94e-06	2.53e+00	3.21e+00	9	1.94e-06	1.38e+00
4.3M	41M	0.084	9	7.66e-07	8.42e+00	9.91e+00	9	7.66e-07	3.62e+00

slightly decreases due to cache effects, this is not the case on the GPU where throughput is still increasing for the very largest problem sizes.

In Table 10, we see a very similar picture for the extruded 3D mesh as for the 2D case, with both implementations achieving equivalent convergence in terms of error and multigrid iterations. Furthermore, the GPU surpasses Broadwell for around 0.5 million DoF and continues to increase the gap all the way to the largest problem size, where a speedup of about 2.6 times is obtained. The Xeon Phi is consistently slower than Broadwell on this test case, which can be explained by its slower performance in the nonvectorized regions of the adapted mesh and suboptimal load balancing between the 128 MPI ranks in the CPU multigrid level operations implemented in `deal.II` [5, 14].

Figure 7 displays the throughput with the largest meshes for $p = 1$ to $p = 4$ in 2D and 3D. The throughput increases with the element degree similarly for both the CPU and the GPU, with the GPU version being about two to three times faster. Approximations with higher-order shape functions deliver the solution more quickly due to faster matrix-vector products as seen also for the uniform mesh cases, and because a higher-order mesh uses fewer elements and thus imposes more structure that is beneficial for the load balancing, particularly for the CPU versions. Figure 8 shows results for the same problems, replacing the Chebyshev (6, 6) smoother by a damped Jacobi smoother with damping parameter derived from the eigenvalue estimate underlying the Chebyshev method, i.e., Chebyshev (1, 1). For this case, the GPU-CPU speedup decreases noticably for low-order elements, whereas the speedup for high-order elements is further in-

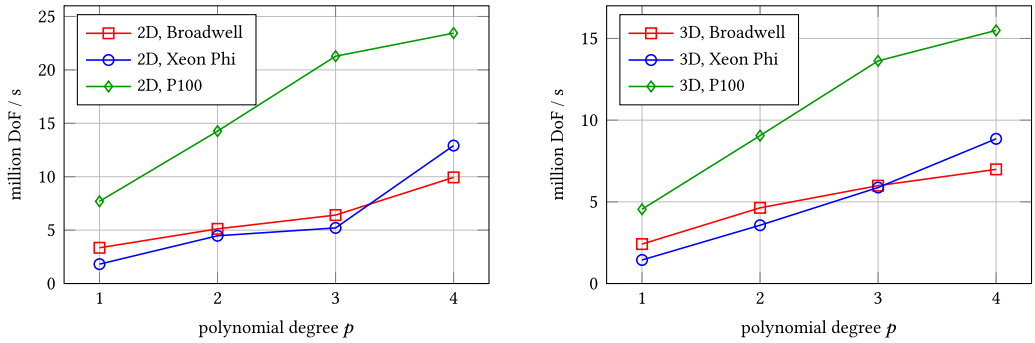


Fig. 7. Throughput of the preconditioned CG solver for the Poisson equation on an L-shaped domain on the largest meshes in 2D (left panel) and 3D (right panel), using between 20 and 30 million DoF for $p = 2, 3, 4$ and about 13 million DoF for $p = 1$ elements. A Chebyshev (6, 6) iteration is used for pre- and postsmoothing on all levels.

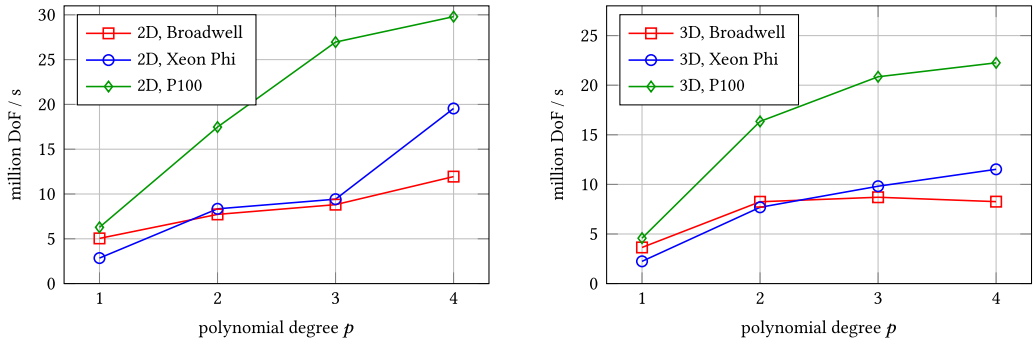


Fig. 8. Throughput for the L-shaped domain on the largest meshes in 2D (left panel) and 3D (right panel). A damped Jacobi smoother is used.

creased, particularly in 3D. This behavior can be explained by the fact that the damped Jacobi smoother goes to coarse levels more often. For higher degrees, we find a larger proportion of elements on the finest level where the GPU can better utilize the parallelism.

5.4 Nonlinear Example

As an example of a nonlinear equation, we consider the solution of the minimal surface equation,[2]

$$-\nabla \cdot \left(\frac{1}{\sqrt{1 + |\nabla u|^2}} \nabla u \right) = 0 \qquad \text{in } \Omega,$$
$$u = g \qquad \text{on } \partial\Omega,$$

which describes the surface of a soap film spanned by a wire loop defining the domain Ω and boundary conditions (e.g., see Ch. 10 in Pinchover and Rubinstein [62]). The nonlinearity is resolved by Newton's iteration, where we solve the following linear PDE for the search direction

[2]This test case is based on Step 15 in the `deal.II` tutorials [5], found at http://www.dealii.org/developer/doxygen/deal.II/step_15.html.

Table 11. Results for the Minimal Surface Equation in 2D With $p = 4$

				Broadwell			P100		
Cells	DoF	Nonlinear Iterations	Residuals	Time Residuals	Linear Iterations	Time Solution	Time Residuals	Linear Iterations	Time Solution
5.1k	82k	12	30	0.00404	91	0.159	0.00248	91	0.727
20k	0.33M	13	32	0.0129	131	0.527	0.00505	129	1.47
82k	1.3M	12	31	0.0618	163	2.77	0.0147	161	3.85
0.33M	5.2M	12	31	0.209	193	17.4	0.0526	186	10.5

δu^n in each step n,

$$-\nabla \cdot \left[\left(\frac{\mathbb{I}}{(1 + |\nabla u^n|^2)^{\frac{1}{2}}} - \frac{(\nabla u^n \otimes \nabla u^n)}{(1 + |\nabla u^n|^2)^{\frac{3}{2}}} \right) \nabla \delta u^n \right] = \nabla \cdot \left(\frac{1}{(1 + |\nabla u^n|^2)^{\frac{1}{2}}} \nabla u^n \right),$$

and then compute the new approximation as

$$u^{n+1} = u^n + \alpha^n \delta u^n .$$

Here, $\alpha^n \in (0, 1]$ is a damping factor determined by a backtracking line search procedure. An initial step with $\alpha_0^n = 1$ (Newton step) is tested and reduced to $\alpha_i^n = \frac{1}{2}\alpha_{i-1}^n$ until $u^n + \alpha_i^n \delta u^n$ does in fact lead to a reduction in the residual norm. The initial guess for the Newton iteration is found by solving the Laplace equation with $\kappa \equiv 1$, which is a good approximation as long as $|\nabla u| \ll 1$. The computational domain is a circular disk of radius 1. As boundary condition, $u = \sin(\pi(x + y))$ is prescribed, which yields a solution depicted in Figure 2(c). The Newton iteration is terminated once the absolute discrete l_2 norm in the nonlinear residual, including the usual finite element volumetric scaling, goes below 10^{-12}. For the linear CG solver, we stop iterating once the unpreconditioned residual has been reduced by 10^4 or once it goes below 10^{-13}, whatever criterion is reached first.

Table 11 lists the results for four successively finer grids using elements with $p = 4$ on both P100 and Broadwell. Here, we record a speedup of 1.66 times for the GPU version, which is somewhat lower than for the linear problems. This can be explained by the overhead in the loop scheduling with CUDA inside the multigrid hierarchy for moderate mesh sizes, which favors the CPU implementation on Broadwell, as detailed in Section 6 and also seen in Table 9. However, the residual computations on the finest level show a speedup of almost 4 times, which is even faster than the two- to threefold speedup for solving the linear systems in the previous sections. This confirms the capabilities of the GPU given enough parallelism and requests a closer analysis of the solver ingredients and the throughput of the matrix-vector product at coarser levels, which is performed in the next section.

The 2D minimal surface equation can readily be extended to three dimensions, in which case we obtain the results in Table 12. Clearly, with the computationally heavier 3D elements, the matrix-vector products for smoothing on the finest level are now the dominant algorithmic component, and the GPU runs up to 3.1 times faster.

6 ANALYSIS OF MULTIGRID PERFORMANCE

The results in the previous section have shown a considerable advantage of the P100 implementation over both the Xeon Phi and the Broadwell CPU for sufficiently large problem sizes. In this section, the multigrid performance is therefore analyzed in more detail in terms of its ingredients.

Table 12. Results for the Minimal Surface Equation Run in 3D With $p = 4$

				Broadwell			P100		
Cells	DoF	Nonlinear Iterations	Residuals	Time Residuals	Linear Iterations	Time Solution	Time Residuals	Linear Iterations	Time Solution
450	30k	8	17	0.00290	23	0.0874	0.00142	23	0.151
3.6k	0.23k	9	20	0.0147	39	0.301	0.00381	40	0.390
29k	1.8M	9	21	0.113	65	3.70	0.0223	68	2.06
0.23M	15M	11	25	0.875	111	50.3	0.198	112	16.2

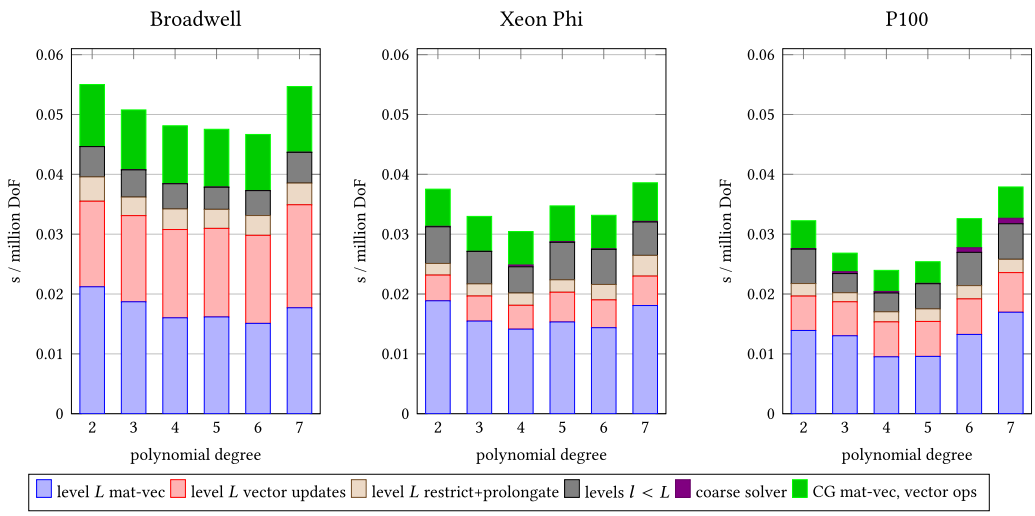


Fig. 9. Breakdown of components of the CG solver preconditioned by a V-cycle with a Chebyshev (4, 4) smoother for the constant-coefficient Laplacian on a 3D Cartesian mesh according to Section 5.1.

6.1 Breakdown of Times

We study the configuration of Section 5.1 and Table 6 with a uniformly meshed Cartesian geometry where both the CPU code and the GPU code can leverage the compression of the Jacobian described in Section 3.1. This moves the code balance from being memory bandwidth bound at around 0.8 to 1.5 floating-point operations per byte loaded toward higher arithmetic intensities of around 4 to 10 Flop/Byte (also see the roofline analysis in Kronbichler and Wall [48]).

Figure 9 shows a breakdown of the contributions to the runtime of the outer CG solver preconditioned by a multigrid V-cycle into the following components:

- the single-precision matrix-vector product of the smoother and residual computation on the finest level $l = L$;
- the single-precision vector operations for residual computation before restriction and after prolongation, as well as the vector updates of the Chebyshev smoother on the finest level $l = L$;
- the multigrid restriction and prolongation operations on the finest level;
- all operations on the intermediate levels except the coarsest and finest one, $1 \leq l < L$;
- the coarse grid solver; and

- the operations in the outer CG solver apart from the preconditioner application, which includes the matrix-vector product in double precision, two inner products, one l_2 norm, and three vector updates for each CG iteration.

Figure 9 reveals several important algorithmic characteristics. First, the vector operations in as simple a method as the Chebyshev iteration take a significant amount of the computing time, particularly on Broadwell. Hardware performance counters show that the fine-level smoother's vector operations run at 110 to 120 GB/s of main memory throughput, i.e., the full memory bandwidth recorded with the STREAM triad benchmark on Broadwell. On the Xeon Phi, the measured throughput during vector updates is up to 420GB/s when using the MCDRAM memory. As a consequence, the Xeon Phi reports only one fourth of the time on vector updates as compared to Broadwell. On the P100, the reported timings for the vector updates are higher. This is because the CPU version uses a tuned version from the `deal.II` library, merging all vector operations of the Chebyshev iteration [1] including the Jacobi preconditioner into a single loop, performing five vector reads and two writes. The GPU falls back to the baseline code of `deal.II` that reads nine vectors and writes five vectors. Taking this difference into consideration, the GPU runs the vector operations at around 550GB/s. The fact that vector operations take a significant share of runtime is a consequence of the high level of performance achieved in the matrix-vector products but is nonetheless remarkable, as focus of the community has previously been mostly on improving the matrix-vector product (e.g., see [45, 46, 48, 63, 68]).

Furthermore, the operations on intermediate levels are cheapest on Broadwell (because they fit into caches) and the P100 and more expensive on the Xeon Phi. The reason for this difference is that the Xeon Phi runs a CPU code with MPI parallelization and vectorization over several cells, where the granularity of domain decomposition and the latency of the MPI data exchange get clearly visible with reduced throughput (also see Figure 11 later). Thus, the CPU threading model is inferior to the more flexible threading with CUDA that can utilize parallelism both within the cells and across cells and offer fast atomic operations.

A contribution to the relatively higher cost of the GPU code on coarser levels and especially the coarse solver can be attributed to latency effects in issuing the CUDA kernels for loops over the cells as well as vector updates. In our GPU code, control returns to the CPU after each call, giving a latency of around 30 to 50 us per call according to our measurements, despite the asynchronous execution on the device. This latency value is larger than on the CPU where the only explicit synchronization points are in the MPI data exchange of matrix-vector products and in inner products. Although a more optimized GPU implementation could hide these effects, as has been done for special cases in the literature [38], these optimizations are not within the scope of the present work where the focus is on a generic C++ implementation with vector interfaces consistent with a large project, in this case the `deal.II` library.

6.2 Detailed Behavior of the Matrix-Vector Product

Finally, we consider the performance of the matrix-vector product. As shown by the performance of the residual evaluation in Section 5.4, as well as the share of matrix-vector products in Figure 9, it can be concluded that P100 performs particularly well. In Figure 10, the throughput of the matrix-free operator evaluation for the Laplacian on both a Cartesian mesh with constant coefficients and on the mesh of the shell example with variable coefficients from Section 5.2 is displayed. The data is based on measuring the time for 1,000 matrix-vector products for the mesh configurations of the cube in Section 5.1 and the shell in Section 5.2, respectively, computing the average runtime, and dividing the number of DoF by the average.

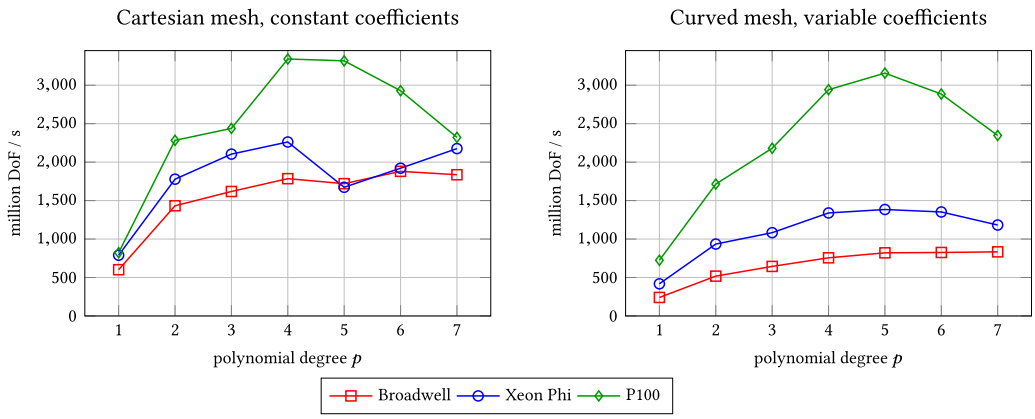


Fig. 10. Throughput of matrix-free evaluation of the Laplacian on a 3D Cartesian mesh (left panel) and shell mesh (right panel).

Table 13. Arithmetic Throughput and Memory Transfer Based on Idealized Access Model of the Matrix-Free Operator Evaluation on the Architectures of Table 4 for the Problem Setups of Tables 6 and 8

		GFLOP/s (percentage of peak)			GB/s (percentage of peak)		
Geometry	Degree	Broadwell	Xeon Phi	P100	Broadwell	Xeon Phi	P100
Cartesian	1	214 (18%)	281 (12%)	424 (9%)	33 (29%)	44 (10%)	180 (30%)
Cartesian	2	273 (23%)	339 (15%)	684 (15%)	54 (47%)	67 (16%)	213 (35%)
Cartesian	3	263 (23%)	341 (15%)	652 (14%)	54 (47%)	70 (17%)	160 (27%)
Cartesian	4	280 (24%)	355 (16%)	892 (19%)	57 (49%)	72 (17%)	181 (30%)
Cartesian	5	274 (24%)	266 (12%)	919 (20%)	53 (46%)	52 (12%)	159 (26%)
Cartesian	6	311 (27%)	318 (14%)	856 (18%)	57 (50%)	58 (14%)	129 (21%)
Cartesian	7	319 (27%)	378 (17%)	721 (15%)	55 (48%)	65 (16%)	96 (16%)
Curved	1	139 (12%)	240 (11%)	356 (8%)	104 (91%)	181 (43%)	430 (72%)
Curved	2	147 (13%)	266 (12%)	497 (11%)	103 (89%)	185 (44%)	434 (72%)
Curved	3	147 (13%)	246 (11%)	568 (12%)	94 (82%)	158 (38%)	389 (65%)
Curved	4	160 (14%)	283 (13%)	768 (16%)	94 (82%)	167 (40%)	432 (72%)
Curved	5	171 (15%)	288 (13%)	858 (18%)	93 (81%)	157 (37%)	410 (68%)
Curved	6	173 (15%)	282 (13%)	829 (18%)	87 (76%)	143 (34%)	344 (57%)
Curved	7	178 (15%)	254 (11%)	719 (15%)	84 (73%)	119 (28%)	264 (44%)

The proposed GPU implementation reaches a throughput of up to 3.4 billion DoF/s on a Cartesian mesh in Figure 10. This number must be compared to Broadwell at up to 1.9 billion DoF/s and the Xeon Phi at up to 2.2 billion DoF/s. The advantage of the GPU implementation is particularly pronounced for moderate polynomial degrees $3 \leq p \leq 6$. Table 13 shows the performance details for these experiments in terms of the achieved arithmetic performance in gigaflops per second and the modeled main memory throughput in gigabytes per second. For the CPU versions, the tensor contractions use the so-called even-odd decomposition [42] that cuts the number of FMAs by half. This is the fastest evaluation option on Broadwell and the Xeon Phi as described in the analysis in Kronbilcher and Kormann [46, Sec. 3.3], despite a lower gigaflops per second rate than plain 1D matrix-vector products. On the GPU, however, this optimization is not used because it involves additional synchronization between lanes that reduces throughput. Thus, for the same DoF per second throughput, a higher gigaflops per second is necessary on the GPU. For memory

transfer, we display the ideal case with perfect cache behavior on the CPUs where shared DoF between neighboring elements need to be loaded only once and the transfer of MPI ghost exchange is neglected. For the GPU, however, the vector data is always accessed from the high-bandwidth memory and modeled accordingly. The performance counter data on CPUs measured with the Likwid tool [70] reports a memory transfer equivalent to around 4.5 vectors, more than the theoretical value of 3 (read input, write output, read-for-ownership on output). This increases the memory throughput by about 20GB/s in the Cartesian mesh case for Broadwell and the Xeon Phi and by around 10GB/s in the curved mesh case. Apart from this deviance, performance counters verify the modeled values. The values on the GPU are in close agreement with results from the `nvprof` tool on the GPU. All three architectures are somewhat away from the theoretical memory bandwidth limits. For example, the indirect addressing for the vector access represents more than half of the instruction counts on the CPU system and also represents around half of the cost on the GPU. Performance counters also report some pipeline stalls due to missing prefetching, even though those are below 10% of all stalls. Note that the instruction mix and data access of the compute part of the tensor contractions with sum factorization has been shown to reach 600GFLOP/s on Broadwell and 1,100GFLOP/s on the Xeon Phi in Kronbichler and Kormann [46], i.e., more than 50% of the arithmetic peak listed in Table 4.

When turning to the curved mesh, Figure 10 displays a lower throughput for all architectures, with up to 3.1 billion DoF/s on the P100, up to 1.4 billion DoF/s on the Xeon Phi, and up to 840 million DoF/s on Broadwell. To put the P100 number in perspective, Świrydowicz et al. [68] report around 400GFLOP/s for evaluating the Laplacian with $p = 5$ with the less accurate collocation case of the Gauss–Lobatto quadrature on a similar P100 hardware. Our result of 860GFLOP/s is due to the additional work due to the basis change for the Gauss quadrature. Throughput and memory transfer numbers are very similar between our work and Świrydowicz et al. [68] because the limiting resource, the memory traffic, is essentially the same.

As can be seen from the performance data in Table 13, Broadwell runs the curved mesh code close to the memory bandwidth limit of 115GB/s reported by the STREAM triad benchmark. Although the simple vector updates allow both the P100 and the Xeon Phi to reach performance near the available bandwidth of 420GB/s on the Xeon Phi and 600GB/s on the P100, this is not the case for the more complicated code flow in the matrix-free evaluation of the Laplacian. The mix of compute-heavy parts of sum factorization and memory-heavy parts for accessing the effective geometry tensor, as well as the vector entries, perform more favorably on the GPU, however, whereas up to 435GB/s or 72% of the STREAM throughput are recorded on the P100; the Xeon Phi only reaches around 44% of the STREAM throughput. Note that Xeon Phis have been reported to reach a high fraction of memory transfer on regular streaming access [16, 44]. A closer analysis of the behavior of the Xeon Phi in the present case reveals that the main cause is insufficient hardware prefetching: The sum factorization algorithm naturally gives rise to blocks of several thousands to tens of thousands of instructions that operate on cached data, followed by operations at quadrature points that access a large stream of data. The third big component in the operator evaluation is indirect addressing steps that again are memory intensive but with a slightly different flavor. We found that the mixture of these three phases together prevents the Xeon Phi from reaching a better memory utilization. The decrease in memory throughput for higher degrees is because the blocks without memory access in sum factorization grow bigger, reducing the overlap of memory access and computations. Conversely, the hardware prefetcher of Broadwell can keep the memory interface busy, albeit at a lower throughput level. Similarly, the P100 can keep more operations from different phases in flight that help memory throughput.

We record a reduced throughput for the GPU with degree $p = 7$ for both the Cartesian mesh and the curved mesh. This can be explained by the sheer number of threads inside a cell, namely

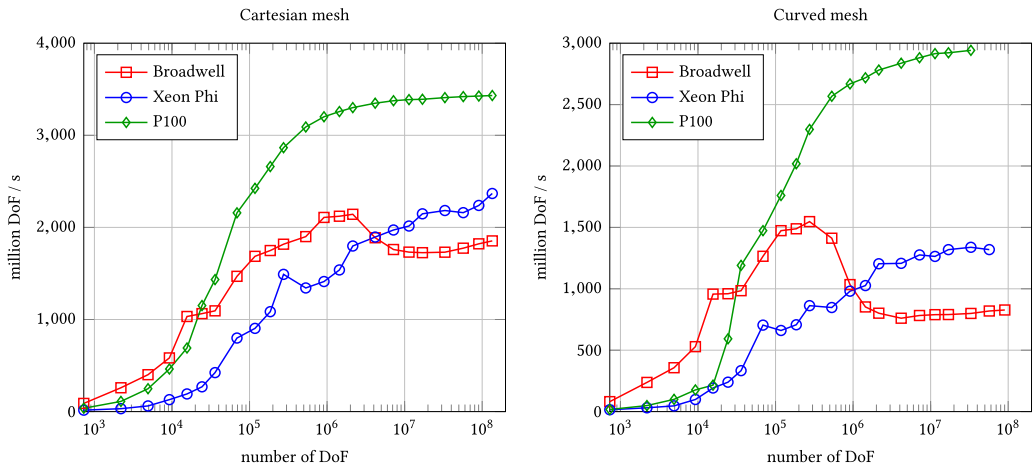


Fig. 11. Throughput of a matrix-vector product for $p = 4$ in 3D.

512, which exceeds the hardware capabilities. Alternative threading strategies that can overcome this limitation are discussed in Świrydowicz et al. [68].

Figure 11 displays the throughput of the matrix-vector product as a function of the problem size, measured as the throughput in millions of DoF per second computed from the average of 1,000 matrix-vector products. A polynomial degree $p = 4$ is chosen for both a Cartesian mesh with geometry compression and a deformed geometry with full access to the tensor $w_q \det(J_q) J_q^{-1} J_q^{-\mathsf{T}}$. The Cartesian mesh tests involve between 729 and 131 million DoF, spanning five orders of magnitude. All three architectures get lower throughput for very small problem sizes where there is not enough parallelism in the problem to saturate the hardware. Surprisingly, the P100 architecture can increase the throughput at a similar pace as the Broadwell CPU, despite the throughput optimization of the former as compared to the latency-driven architecture of Broadwell. This can be explained by the threading within cells on the GPU. Furthermore, the cost of MPI pack, unpack, and `memcpy` operations on Broadwell are significant up to 1 million DoF. At sizes around 10^6 DoF on the Cartesian mesh and $4 \cdot 10^5$ DoF on the curved mesh, Broadwell reaches best throughput because all data fits into the fast level 3 cache, whereas most of the vector and the geometry data must be fetched from main memory beyond that size. Both Broadwell and the Xeon Phi show a slight increase in throughput at large sizes because the MPI ghost exchange, which happens on surfaces around the 28 and 128 subdomains, respectively, gets cheaper as the volume-to-surface ratio increases. On the GPU, the throughput increases smoothly, reaching high throughput surprisingly early, again highlighting the benefits of the chosen implementation, the hardware capabilities, and the flexibility of CUDA for this type of algorithm.

Finally, Figure 12 shows the throughput of the same experiment as a function of the time per matrix-vector product, highlighting the latency barriers of the various architectures. As expected, Broadwell achieves the lowest latency of around 10us, ahead of the P100 with around 30us. The Xeon Phi, however, is not able to decrease runtime below 100us, which is clearly showing the limits of the MPI implementation in conjunction with the Phi's hardware properties. This higher latency is responsible for the reduced efficiency of the Xeon Phi for the FMG cycle of the shell example of Table 8 for high polynomial degrees, as the cost of coarser multigrid levels add up and cannot be neglected any more. We note that latency barriers significantly increase for the MPI version once transfer over InfiniBand fabrics is involved. Kronbilchler and Wall [48] showed a latency barrier at around 100us.

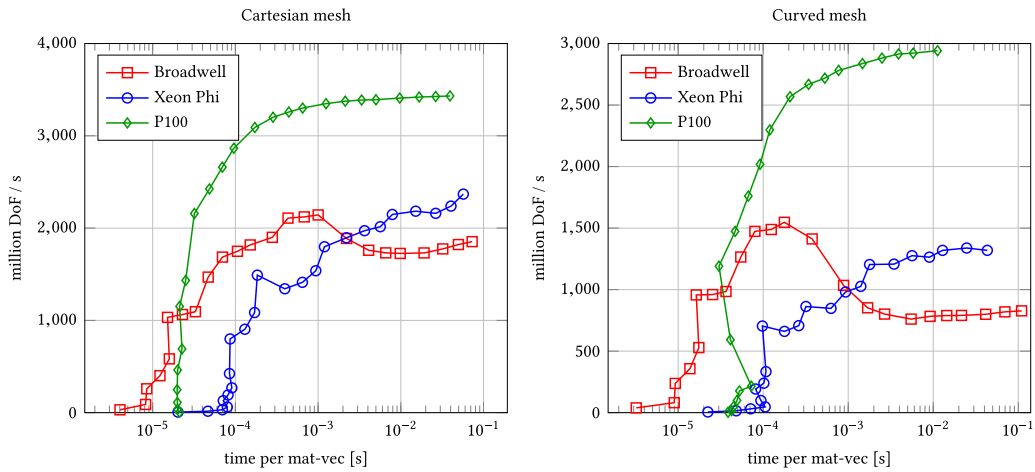


Fig. 12. Throughput of the matrix-vector product over the time for its evaluation for $p = 4$ in 3D.

7 CONCLUSION

We have presented a high-performance implementation of a geometric multigrid solver targeting the matrix-free finite-element solution of Poisson's equation on GPUs for higher-order polynomials on hexahedral meshes. We have demonstrated excellent node-level performance across a range of numerical test cases, involving non-Cartesian geometries, adaptive meshes with hanging nodes, and a nonlinear Newton iteration. We have compared our GPU implementation running on an Nvidia Tesla P100 against a highly optimized CPU version running on a 28-core Intel Broadwell setup and a 64-core Knights Landing Xeon Phi with similar wattage, and found the GPU implementation to be consistently faster except deep in the strong scaling limit, reaching between two and three times better throughput than Broadwell and two times better throughput than the Xeon Phi. Furthermore, we have demonstrated that a mixed precision approach can be used successfully within the multigrid framework to get speedups of up to 83%. In addition, we have shown that, on the Pascal architecture, atomic operations allow to implement matrix-vector products faster than a graph coloring approach for performing the reduction of element contributions, in contrast to previous GPU architectures. The new multigrid algorithms are particularly efficient for higher-order discretizations, achieving best performance in terms of throughput of DoF per second for polynomial degrees between 3 and 5.

A detailed performance analysis has been performed that has highlighted the beneficial features of the GPU regarding the matrix-free operator evaluation with sum factorization as compared to the CPU versions. On the one hand, the CUDA execution model allows to parallelize within elements. Furthermore, the P100 delivers performance closer to the hardware limits than the Xeon Phi for kernels that both stress the memory interface and involve heavy arithmetics, such as our experiments on curved meshes. In particular, we have recorded high throughput of the matrix-vector product already for moderate problem sizes of around 1 million DoF. It takes as little as 288 us to evaluate the 3D Laplacian on fourth-degree hexahedral elements with 912k DoF. In our analysis, the matrix-vector products have been found to be well optimized to the extent that the vector updates in the Chebyshev smoother take a significant time not only on a Broadwell CPU with its slower access to main memory but also on P100 with high-bandwidth memory. Thus, application codes relying on complex solvers need to put focus not solely on the matrix-vector product but also on the whole solver stack including the vector operations, where it is beneficial

to optimize for data access and merge loop to reduce the data access and avoid implicit or explicit barriers in the implementation.

ACKNOWLEDGMENTS

The authors thank the reviewers for their valuable comments that improved the article. Furthermore, discussions with Jed Brown regarding the HPGMG-FE solver are acknowledged. The authors acknowledge the support and hardware access through the Bayerische Kompetenznetzwerk für Technisch-Wissenschaftliches Hoch- und Höchstleistungsrechnen (KONWIHR) in the framework of the project *Matrix-free GPU kernels for complex applications in fluid dynamics*. This work was supported by the German Research Foundation (DFG) under the project *High-order discontinuous Galerkin for the exa-scale* (ExaDG) within the priority program *Software for Exascale Computing* (SPPEXA), grant agreement no. KR4661/2-1. The Tesla K40 experiments were run on hardware provided by the Linnaeus center of excellence UPMARC, Uppsala Programming for Multicore Architectures Research Center.

REFERENCES

[1] Mark Adams, Marian Brezina, Jonathan Hu, and Ray Tuminaro. 2003. Parallel multigrid smoothing: Polynomial versus Gauss–Seidel. *Journal of Computational Physics* 188 (2003), 593–610. DOI: https://doi.org/10.1016/S0021-9991(03)00194-3

[2] Mark Adams, Jed Brown, John Shalf, Brian Van Straalen, Erich Strohmaier, and Samuel Williams. 2019. High-performance geometric multigrid. Retrieved January 18, 2019 from https://hpgmg.org/.

[3] Mark Adams, Phillip Colella, Daniel T. Graves, Jeff N. Johnson, Hans S. Johansen, Noel D. Keen, Terry J. Ligocki, et al. 2015. *Chombo Software Package for AMR Applications Design Document*. Technical Report. Lawrence Berkeley National Laboratory. https://crd.lbl.gov/assets/pubs_presos/chomboDesign.pdf.

[4] Mark F. Adams, Ravi Samtaney, and Achi Brandt. 2010. Toward textbook multigrid efficiency for fully implicit resistive magnetohydrodynamics. *Journal of Computational Physics* 229, 18 (2010), 6208–6219. DOI: https://doi.org/10.1016/j.jcp.2010.04.024

[5] Giovanni Alzetta, Daniel Arndt, Wolfgang Bangerth, Vishal Boddu, Benjamin Brands, Denis Davydov, Rene Gassmöller, et al. 2018. The `deal.II` library, version 9.0. *Journal of Numerical Mathematics* 26, 4 (2018), 173–184. DOI: https://doi.org/10.1515/jnma-2018-0054

[6] Wolfgang Bangerth, Carsten Burstedde, Timo Heister, and Martin Kronbichler. 2011. Algorithms and data structures for massively parallel generic finite element codes. *ACM Transactions on Mathematical Software* 38, 2 (2011), Article 14. DOI: https://doi.org/10.1145/2049673.2049678

[7] Marsha J. Berger, Michael J. Aftosmis, and Gedas Adomavicius. 2001. Parallel multigrid on Cartesian meshes with complex geometry. In *Parallel Computational Fluid Dynamics*. North Holland, Amsterdam, 283–290.

[8] Achi Brandt. 1977. Multi-level adaptive solutions to boundary-value problems. *Mathematics of Computation* 31 (1977), 333–390. DOI: https://doi.org/10.1090/S0025-5718-1977-0431719-X

[9] Jed Brown. 2010. Efficient nonlinear solvers for nodal high-order finite elements in 3D. *Journal of Scientific Computing* 45, 1–3 (2010), 48–63. DOI: https://doi.org/10.1007/s10915-010-9396-8

[10] Jed Brown, Veselin Dobrev, Som Dutta, Paul Fischer, Kazem Kamran, Tzanio Kolev, David Medina, et al. 2018. *ECP Milestone Report: Propose High-Order Mesh/Data Format, WBS 2.2.6.06, Milestone CEED-MS18*. Technical Report. U.S. Department of Energy. https://ceed.exascaleproject.org/docs/ceed-ms18-report.pdf.

[11] Jed Brown, Barry Smith, and Aron Ahmadia. 2013. Achieving textbook multigrid efficiency for hydrostatic ice sheet flow. *SIAM Journal on Scientific Computing* 35, 2 (2013), B359–B375. DOI: https://doi.org/10.1137/110834512

[12] CEED. 2018. CEED Bake-Off Problems (Benchmarks). Retrieved April 8, 2019 from http://ceed.exascaleproject.org/bps/.

[13] Justin Chang, Maurice S. Fabien, Matthew G. Knepley, and Richard T. Mills. 2018. Comparative study of finite element methods using the time-accuracy-size (TAS) spectrum analysis. *SIAM Journal on Scientific Computing* 40, 6 (2018), C779–C802. DOI: https://doi.org/10.1137/18M1172260

[14] Thomas C. Clevenger, Timo Heister, Guido Kanschat, and Martin Kronbichler. 2019. A flexible, parallel, adaptive geometric multigrid method for FEM. arXiv:1904.03317 preprint.

[15] Michel O. Deville, Paul F. Fischer, and Ernest H. Mund. 2002. *High-Order Methods for Incompressible Fluid Flow*. Vol. 9. Cambridge University Press.

[16] Maurice S. Fabien, Matthew G. Knepley, Richard T. Mills, and Beatrice M. Rivière. 2019. Manycore parallel computing for a hybridizable discontinuous Galerkin nested multigrid method. *SIAM Journal on Scientific Computing* 41, 2 (2019), C73-C96. DOI: https://doi.org/10.1137/17M112890

[17] Niklas Fehn, Wolfgang A. Wall, and Martin Kronbichler. 2018. Efficiency of high-performance discontinuous Galerkin spectral element methods for under-resolved turbulent incompressible flows. *International Journal for Numerical Methods in Fluids* 88, 1 (2018), 32–54. DOI: https://doi.org/10.1002/fld.4511

[18] Niklas Fehn, Wolfgang A. Wall, and Martin Kronbichler. 2019. A matrix-free high-order discontinuous Galerkin compressible Navier–Stokes solver: A performance comparison of compressible and incompressible formulations for turbulent incompressible flows. *International Journal for Numerical Methods in Fluids* 89, 3 (2019), 71–102. DOI: https://doi.org/10.1002/fld.4683

[19] Chunsheng Feng, Shi Shu, Jinchao Xu, and Chen-Song Zhang. 2014. Numerical study of geometric multigrid methods on CPU-GPU heterogeneous computers. *Advances in Applied Mathematics and Mechanics* 6, 1 (2014), 1–23. DOI: https://doi.org/10.1017/S2070073300002411

[20] Krzysztof J. Fidkowski, Todd A. Oliver, James Lu, and David L. Darmofal. 2005. p-multigrid solver of high-order discontinuous Galerkin discretizations of the compressible Navier–Stokes equations. *Journal of Computational Physics* 207, 1 (2005), 92–113.

[21] Paul F. Fischer, Thilina Rathnayake, Som Dutta, Veselin Dobrev, Tzanio Kolev, Jean-Sylvain Camier, Martin Kronbichler, et al. 2019. Running faster in HPC applications. In Preparation.

[22] Markus Geveler, Dirk Ribbrock, Dominik Göddeke, Peter Zajac, and Stefan Turek. 2013. Towards a complete FEM-based simulation toolkit on GPUs: Unstructured grid finite element geometric multigrid solvers with strong smoothers based on sparse approximate inverses. *Computers & Fluids* 80 (2013), 327–332. DOI: https://doi.org/10.1016/j.compfluid.2012.01.025

[23] Amir Gholami, Dhairya Malhotra, Hari Sundar, and George Biros. 2016. FFT, FMM, or multigrid? A comparative study of state-of-the-art Poisson solvers for uniform and nonuniform grids in the unit cube. *SIAM Journal on Scientific Computing* 38, 3 (2016), C280–C306. DOI: https://doi.org/10.1137/15M1010798

[24] Bjorn Gmeiner, Ulrich Rüde, Holger Stengel, Christian Waluga, and Barbara Wohlmuth. 2015. Performance and scalability of hierarchical hybrid multigrid solvers for stokes systems. *SIAM Journal on Scientific Computing* 37, 2 (2015), C143–C168. DOI: https://doi.org/10.1137/130941353

[25] Dominik Göddeke, Robert Strzodka, and Stefan Turek. 2007. Performance and accuracy of hardware-oriented native-, emulated-and mixed-precision solvers in FEM simulations. *International Journal of Parallel Emergent and Distributed Systems* 22, 4 (2007), 221–256. DOI: https://doi.org/10.1080/17445760601122076

[26] William D. Gropp, Dinesh K. Kaushik, David E. Keyes, and Barry F. Smith. 1999. Towards realistic performance bounds for implicit CFD codes. In *Proceedings of Parallel CFD'99*.

[27] William D. Gropp, Dinesh K. Kaushik, David E. Keyes, and Barry F. Smith. 2000. Performance modeling and tuning of an unstructured mesh CFD application. In *SC'00: Proceedings of the 2000 ACM/IEEE Conference on Supercomputing*. 34. DOI: https://doi.org/10.1109/SC.2000.10059

[28] Mark Harris. 2007. Optimizing CUDA. In *SC'07: High Performance Computing With CUDA*. Nvidia.

[29] Brian Helenbrook, Dimitri J. Mavriplis, and Harold L. Atkins. 2003. Analysis of p-multigrid for continuous and discontinuous finite element discretizations. In *Proceedings of the 16th AIAA Computational Fluid Dynamics Conference*.

[30] Michael A. Heroux, Roscoe A. Bartlett, Vicki E. Howle, Robert J. Hoekstra, Jonathan J. Hu, Tamara G. Kolda, Richard B. Lehoucq, et al. 2005. An overview of the Trilinos project. *ACM Transactions on Mathematical Software* 31, 3 (2005), 397–423. DOI: https://doi.org/10.1145/1089014.1089021

[31] Bärbel Janssen and Guido Kanschat. 2011. Adaptive multilevel methods with local smoothing for Hˆ1-and Hˆcurl-conforming high order finite element methods. *SIAM Journal on Scientific Computing* 33, 4 (2011), 2095–2114. DOI: https://doi.org/10.1137/090778523

[32] James Jeffers, James Reinders, and Avinash Sodani. 2016. *Intel Xeon Phi Processor High Performance Programming, Knights Landing Edition*. Morgan Kaufmann, Cambridge, MA.

[33] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P. Scarpazza. 2018. *Dissecting the Volta GPU Architecture Through Microbenchmarking: GTC 2018*. Retrieved April 8, 2019 from http://on-demand.gputechconf.com/gtc/2018/presentation/s8122-dissecting-the-volta-gpu-architecture-through-microbenchmarking.pdf.

[34] Alison C. Jones and Peter K. Jimack. 2005. An adaptive multigrid tool for elliptic and parabolic systems. *International Journal for Numerical Methods in Fluids* 47 (2005), 1123–1128. DOI: https://doi.org/10.1002/fld.845

[35] Guido Kanschat. 2004. Multilevel methods for discontinuous Galerkin FEM on locally refined meshes. *Computers & Structures* 82, 28 (2004), 2437–2445. DOI: https://doi.org/10.1016/j.compstruc.2004.04.015

[36] George E. Karniadakis and Spencer J. Sherwin. 2013. *Spectral/HP Element Methods for Computational Fluid Dynamics*. Oxford University Press. DOI: https://doi.org/10.1093/acprof:oso/9780198528692.001.0001

[37] Donald W. Kelly, Jorge P. De Sousa Rodrigues Gago, Olgierd C. Zienkiewicz, and Ivo Babuška. 1983. A posteriori error analysis and adaptive processes in the finite element method: Part I—Error analysis. *International Journal for Numerical Methods in Engineering* 19 (1983), 1593–1619. DOI: https://doi.org/10.1002/nme.1620191103

[38] Andreas Klöckner. 2014. Loo.py: Transformation-based code generation for GPUs and CPUs. In *Proceedings of ARRAY'14: ACM SIGPLAN Workshop on Libraries, Languages, and Compilers for Array Programming*. DOI: https://doi.org/10.1145/2627373.2627387

[39] Andreas Klöckner, Tim Warburton, Jeffrey Bridge, and Jan S. Hesthaven. 2009. Nodal discontinuous Galerkin methods on graphics processors. *Journal of Computational Physics* 228, 21 (2009), 7863–7882. DOI: https://doi.org/10.1016/j.jcp.2009.06.041

[40] Matthew G. Knepley and Andy R. Terrel. 2013. Finite element integration on GPUs. *ACM Transactions on Mathematical Software* 39, 2 (Feb. 2013), Article 10, 13 pages. DOI: https://doi.org/10.1145/2427023.2427027

[41] Dimitri Komatitsch, Gordon Erlebacher, Dominik Göddeke, and David Michéa. 2010. High-order finite-element seismic wave propagation modeling with MPI on a large GPU cluster. *Journal of Computational Physics* 229, 20 (2010), 7692–7714. DOI: https://doi.org/10.1016/j.jcp.2010.06.024

[42] David A. Kopriva. 2009. *Implementing Spectral Methods for Partial Differential Equations: Algorithms for Scientists and Engineers*. Springer.

[43] Katharina Kormann and Martin Kronbichler. 2011. Parallel finite element operator application: Graph partitioning and coloring. In *Proceedings of the 7th IEEE International Conference on eScience*. 332–339. DOI: https://doi.org/10.1109/eScience.2011.53

[44] Moritz Kreutzer, Georg Hager, Gerhard Wellein, Holger Fehske, and Alan R. Bishop. 2014. A unified sparse matrix data format for efficient general sparse matrix-vector multiplication on modern processors with wide SIMD units. *SIAM Journal on Scientific Computing* 36, 5 (2014), C401–C423. DOI: https://doi.org/10.1137/130930352

[45] Martin Kronbichler and Katharina Kormann. 2012. A generic interface for parallel cell-based finite element operator application. *Computers & Fluids* 63 (2012), 135–147. DOI: https://doi.org/10.1016/j.compfluid.2012.04.012

[46] Martin Kronbichler and Katharina Kormann. 2019. Fast matrix-free evaluation of discontinuous Galerkin finite element operators. *ACM Transactions on Mathematical Software*, in press (2019). DOI: https://doi.org/10.1145/3325864

[47] Martin Kronbichler, Katharina Kormann, Igor Pasichnyk, and Momme Allalen. 2017. Fast matrix-free discontinuous Galerkin kernels on modern computer architectures. In *ISC High Performance*. Lecture Notes in Computer Science, Vol. 10266. Springer, 237–255. DOI: https://doi.org/10.1007/978-3-319-58667-0 13

[48] Martin Kronbichler and Wolfgang A. Wall. 2018. A performance comparison of continuous and discontinuous Galerkin methods with fast multigrid solvers. *SIAM Journal on Scientific Computing* 40, 5 (2018), A3423–A3448. DOI: https://doi.org/10.1137/16M110455X

[49] Karl Ljungkvist. 2014. Matrix-free finite-element operator application on graphics processing units. In *Euro-Par 2014: Parallel Processing Workshops*. Lecture Notes in Computer Science, Vol. 8806. Springer, 450–461. DOI: https://doi.org/10.1007/978-3-319-14313-2_38

[50] Karl Ljungkvist. 2017. Matrix-free finite-element computations on graphics processors with adaptively refined unstructured meshes. In *HPC'17: Proceedings of the 25th High Performance Computing Symposium*.

[51] James W. Lottes and Paul F. Fischer. 2005. Hybrid multigrid-Schwarz algorithms for the spectral element method. *Journal of Scientific Computing* 24 (2005), 613–646. DOI: https://doi.org/10.1007/s10915-004-4787-3

[52] Hong Luo, Joseph D. Baum, and Rainald Löhner. 2006. A p-multigrid discontinuous Galerkin method for the Euler equations on unstructured grids. *Journal of Computational Physics* 211 (2006), 767–783. DOI: https://doi.org/10.1016/j.jcp.2005.06.019

[53] Robert E. Lynch, John R. Rice, and Donald H. Thomas. 1964. Direct solution of partial difference equations by tensor product methods. *Numerische Mathematik* 6 (1964), 185–199. DOI: https://doi.org/10.1007/BF01386067

[54] Dave A. May, Jed Brown, and Laetitia Le Pourhiet. 2014. pTatin3D: High-performance methods for long-term lithospheric dynamics. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC'14)*. 274–284. DOI: https://doi.org/10.1109/SC.2014.28

[55] Stephen McCormick and James Thomas. 1986. The fast adaptive composite grid (FAC) method for elliptic equations. *Mathematics of Computation* 46, 174 (1986), 439–456. DOI: https://doi.org/10.2307/2007986

[56] Eike Müller, Xu Guo, Robert Scheichl, and Sinan Shi. 2013. Matrix-free GPU implementation of a preconditioned conjugate gradient solver for anisotropic elliptic PDEs. *Computing and Visualization in Science* 16, 2 (2013), 41–58. DOI: https://doi.org/10.1007/s00791-014-0223-x

[57] Nvidia Corporation. 2013. *CUDA cuSPARSE Library*. Version 8.0. Nvidia.

[58] Nvidia Corporation. 2016. *GP100 Pascal Whitepaper*. Technical Report. Version 1.1. Nvidia.

[59] Nvidia Corporation. 2016. *Kepler Tuning Guide*. Nvidia.

[60] Steven A. Orszag. 1980. Spectral methods for problems in complex geometries. *Journal of Computational Physics* 37, 1 (1980), 70–92. DOI: https://doi.org/10.1016/0021-9991(80)90005-4

[61] Anthony T. Patera. 1984. A spectral element method for fluid dynamics: Laminar flow in a channel expansion. *Journal of Computational Physics* 54, 3 (1984), 468–488. DOI: https://doi.org/10.1016/0021-9991(84)90128-1

[62] Yehuda Pinchover and Jacob Rubinstein. 2005. *An Introduction to Partial Differential Equations*. Cambridge University Press.

[63] Jean-Francois Remacle, Rajesh Gandham, and Timothy Warburton. 2016. GPU accelerated spectral finite elements on all-hex meshes. *Journal of Computational Physics* 324 (2016), 246–257. DOI: https://doi.org/10.1016/j.jcp.2016.08.005

[64] Johann Rudi, A. Cristiano I. Malossi, Tobin Isaac, Georg Stadler, Michael Gurnis, Peter W. J. Staar, Yves Ineichen, et al. 2015. An extreme-scale implicit solver for complex PDEs: Highly heterogeneous flow in Earth's mantle. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC'15)*. 5:1–5:11. DOI: https://doi.org/10.1145/2807591.2807675

[65] Jörg Stiller. 2017. Nonuniformly weighted Schwarz smoothers for spectral element multigrid. *Journal of Scientific Computing* 72 (2017), 81–96. DOI: https://doi.org/10.1007/s10915-016-0345-z

[66] Hari Sundar, George Biros, Carsten Burstedde, Johann Rudi, Omar Ghattas, and Georg Stadler. 2012. Parallel geometric-algebraic multigrid on unstructured forests of octrees. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC'12)*. DOI: https://doi.org/10.1109/SC.2012.91

[67] Hari Sundar, Georg Stadler, and George Biros. 2015. Comparison of multigrid algorithms for high-order continuous finite element discretizations. *Numerical Linear Algebra With Applications* 22 (2015), 664–680. DOI: https://doi.org/10.1002/nla.1979

[68] Kasia Świrydowicz, Noel Chalmers, Ali Karakus, and Timothy Warburton. 2019. Acceleration of tensor-product operations for high-order finite element methods. *International Journal for High Performance Computing Applications*. In Press. DOI: https://doi.org/10.1177/1094342018816368

[69] James L. Thomas, Boris Diskin, and Achi Brandt. 2003. Textbook multigrid efficiency for fluid simulations. *Annual Reviews of Fluid Mechanics* 35 (2003), 317–340.

[70] Jan Treibig, Georg Hager, and Gerhard Wellein. 2010. LIKWID: A lightweight performance-oriented tool suite for x86 multicore environments. In *Proceedings of PSTI'10: The 1st International Workshop on Parallel Software Tools and Tool Infrastructures*. DOI: https://doi.org/10.1109/ICPPW.2010.38 https://github.com/RRZE-HPC/likwid.

[71] Ulrich Trottenberg, Cornelius Oosterlee, and Anton Schüller. 2001. *Multigrid*. Elsevier Academic Press, London, UK.

[72] Henry M. Tufo and Paul F. Fischer. 1999. Terascale spectral element algorithms and implementations. In *Proceedings of the 1999 ACM/IEEE Conference on Supercomputing*. ACM, New York, NY.

[73] Bruno Turcksin, Martin Kronbichler, and Wolfgang Bangerth. 2016. *WorkStream*—A design pattern for multicore-enabled finite element computations. *ACM Transactions on Mathematical Software* 43, 1 (2016), Article 2, 29 pages. DOI: https://doi.org/10.1145/2851488

[74] Vasily Volkov. 2010. Better performance at lower occupancy. In *Proceedings of the GPU Technology Conference (GTC'10)*, Vol. 10. 16.

[75] Nicholas Wilt. 2013. *The CUDA Handbook: A Comprehensive Guide to GPU Programming*. Pearson Education.

Received January 2018; revised February 2019; accepted February 2019